Findings: experiments, confirmations and errors

Physicomimetics in BPP: what the conversion has found, kept up to date · 3 November 2026

A running record of what the conversion has found: the book's results we have reproduced, new results, errors and discrepancies in the book and in the other implementations, and our own mistakes, corrected. It is updated with every patch that finds something, and `FINDINGS.pdf` beside it is rebuilt each time. `BOOK.md` says what to build and `FIXES.md` what was fixed in the code; this file says what was learned.

How to read an entry. Each has an ID, the date, what was done, the numbers, where the evidence lives (a check script, a README or a probe), and a status: **Confirmed** (the book's claim, reproduced by our own measurement), **New** (a result the book does not have), **Book error** (the book disagrees with its own code, figures or arithmetic), **Book discrepancy** (the same, but small or an omission, not changing the conclusion), **Divergence** (NetLogo, the Android app and BPP differ), **Our error** (something we got wrong, and how it was found and corrected) or **Open** (measured but not yet settled).

Page numbers are PDF pages of `apbook1.pdf`; book page = PDF − 31.

1. The book's results, reproduced

F1. Appendix A: glibc `random()` is biased, and BPP uses it — 2026-10-17, Confirmed. `checks/rng_appendix.lua`. BPP's `math.random` is glibc `rand()` = `random()`, in Lua 5.1 and 5.2 alike (William's BPP links 5.2, the cloud build 5.1). Run with Listing A.1's sampling (S extra calls per accepted point in the unit disk, 10^6 points, 36 bins, 99.9% cut χ^2 66.6), it fails at S = 25 (χ^2 321) and 27 (409), as the book shows (PDF 646–648); Lua 5.2 the same (321, 410); also S = 26 (223) and 3 (70). The multiply-with-carry generator now in `lib/rng.lua` passes at every S tried (χ^2 26–47).

F2. Ch. 2.1 spring1D, every result — 2026-10-17, Confirmed. `checks/ch02.lua`, `ch02/README.md`. Momentum exactly 0; the separation oscillates symmetrically about D (27.000 to 73.000); period ×√10 at k = 1 (1.405 and 4.443); the centre of mass sits by the heavy mass at ratio 10; friction 0.001 drains the energy (2641 to 0.016) and the separation settles at D; changing D mid-run moves the energy to a new constant level (2641 to 998).

F3. Ch. 2.2 spring2D: angular momentum and the centrifugal stretch — 2026-10-17, Confirmed. `checks/ch02.lua`. Angular momentum constant to 2×10^{-14}, linear momentum 10^{-12}; mean separation 77.8 > D = 50; the maximum exceeds 2D = 100 for every seed tried (111–118), with spring2D.nlogo's random start (F41).

F4. Ch. 2.3 solar_system, experiments a–e — 2026-10-17, Confirmed. `checks/ch02.lua`. (a) A lighter sun gives a larger orbit, more so for AM 15 (87 to 133) than AM 10. (b) At Δt 0.1, AM 5 escapes (E from −121 to +111) and AM 0 is nonsense (E to 448,804). (c) At Δt 0.001, AM 5 orbits properly and AM 0 still shoots off left (x = −36,312). (d) Velocity caps fail: Vmax 9 drops the planet into the sun (|P| up to 891). (e) Force cap Fmax 1: bound, precessing rosettes (AM 4 precesses 112° per orbit); AM 0 swings through the sun, 0 to 15.6 away.

F5. Ch. 2.3e: the book's peak speeds are right — 2026-10-17, Confirmed. `checks/ch02.lua`. The text's "5.6, 5, 4 and 9 (when AM is zero, one, four and nine)" (PDF 84–85) looked non-monotone, and `book/ch02.md` advised re-measuring. We get 5.50, 4.97, 4.00, 9.00. For AM 4 and 9 the start is the closest point, so the peak is the starting speed; AM 0 falls straight through the sun and AM 1 nearly does, so both peak near the centre.

F6. Ch. 11.3: the mean free path 0.651757, and 0.6079 with a buffer — 2026-10-17, Confirmed. `ch11/README.md`, `checks/mfp_theory.lua`. The integral, computed to 60 digits, is 0.6517567914559… (Eq. 11.4 gives 0.651757); the 5% buffer gives 0.607939 (PDF 383: 0.6079). Listing 11.4 at 10^7 samples agrees within one standard error.

F7. Ch. 11.2: coverage converges to a seed-independent limit — 2026-10-17, Confirmed. `HANDOFF.md` §6. As the book argues from ergodicity (PDF 380), long runs from different seeds reach the same deviation: at MFP 50 / 100 with one agent, 0.0158 / 0.0368 from both seeds to within 1%.

F8. Fig. 4.19: 100 agents cover more evenly than 1 at large mean free paths — 2026-10-17, Confirmed. At equal visits (5×10^7), MFP 30 / 50 give 0.0035 / 0.0158 with one agent and 0.00087 / 0.0035 with 100. Crowding adds turns, shortening the effective free path. Agents are near-independent only when they rarely meet.

F43. Ch. 13.2: the split-Newtonian point mass dithers across D — 2026-10-18, Confirmed (E2, PDF 447–449). `checks/ch13.lua`, `probes/settle_drones.lua`. The book says the sign flip at D lets a point mass dither across it with almost no net motion. The 2D particle replica of the drone law (G 1500, power 2.2, forceMax 1.5, D 50, friction 0.6, 40 particles) does exactly that: after 1,500 ticks the bonds are at 0.99 D but every particle still moves 0.55 units per tick. Flown by 50 drones the dither becomes real motion that never stops: 0.27 m/s (2D) and 0.47 m/s (3D) after 170 s, 190 and 324 AR-boundary crossings per drone per minute, and a suggested-position gap of 6 and 11 cm (3 seeds each). Lennard-Jones on the same drones settles in 20 s (2D) and at the last launch wave (3D).

F44. Ch. 13.5: AR damping η settles the Newtonian drones like LJ — 2026-10-18, Confirmed on drones (the book's Pioneers settle, PDF 454). `quadcopter/README.md` ("Ch. 13 on the drones"). With η (Eq. 13.9, power 1) and nothing else, the same 50 drones settle (scalar acceleration below 0.1 m/s² from then on; but see F52 for when the lattice is formed) 6 s after AP engages in 2D and 29 s in 3D, when the last launch wave joins (LJ: 20 s and 31–33 s); AR crossings fall to 0–1 per drone per minute, speed to 0.001–0.003 m/s, the gap to 0.03–0.09 mm, with no crashes (baseline 2 per 2D run). This answers `BOOK.md` §1 item 1's expectation: yes, η alone does it, because the law is then smooth through zero at D. The square and cube formations settle too: 6 s (2D square, LJ 23–25 s) and 27 s (cube, LJ 33 s), against 0.33 and 0.63 m/s of endless jitter without η; LJ's cube is the more regular (bond CV 0.007–0.03 against η's 0.07–0.09). The lattice η settles into is the subject of F45.

F73. Ch. 3.2.2: the clustering transition G_t, its hysteresis, and G_t in 3D — 2026-10-25, Confirmed (2D) and New (3D). `ch03/README.md`, `checks/ch03.lua`. On a line-for-line copy of `formation_newton.nlogo` (`ch03/models.lua`) at the book's settings (G 400, p 2, D 30, friction 0.5, Δt 1, 50 particles): Fmax 1 clusters (21–24 nodes; the book's Fig. 3.16 has 21), Fmax 5 does not; from the starting cloud nothing clusters at 0.9 G_t and 68–80% of the particles do at 1.2 G_t; lowering G slowly from a clustered lattice, the clusters loosen and then break up at 0.94–1.00 G_t (all but two pairs, 6 seeds), the last pairs, held by lattice defects with more than six neighbours, at 0.80–0.96 G_t. Raising G on a formed lattice makes 3 pairs at most; raising it and adding 10 particles clusters 33–43%; removing 25 of 50 clustered particles leaves 16–21 of 21–22 nodes (the book: 19 of 21). Eq. 3.4 is exact for the model's triangle. *3D (ours):* the book's argument (the ring around a cluster pushes a displaced particle back with G/D^p per neighbour, projected on the displacement; the cluster holds while that beats Fmax in its weakest direction) gives, for close packing's 12 neighbours, **G_t3 = Fmax D^p / (2√6) = G_t / √2**; measured with a 1.3 D view, the clusters break up at 0.92–1.12 G_t3. The book's 1.5 D view reaches close packing's second shell (√2 D) and shrinks the 3D lattice to 0.64 D. The BPP scene `Formations-Newtonian.lua` at the book's sliders shows the same transition (none at 0.9 G_t, 62–74% at 1.2 G_t, 19–22 nodes at G 400; `probes/ch03_scene.lua`); its own defaults are below their G_t in 2D and 3D, so it does not cluster.

F75. Ch. 3.2.1 and 3.3: Hooke's law, the goal and obstacles reproduce; Disable One's threshold is a bound, not the threshold — 2026-10-25, Confirmed and New. `ch03/README.md`, `checks/ch03.lua`. Hooke's law at the book's settings forms a good lattice by 200 steps (nn 0.95 D, then 0.99 D and still); Fmax 3 cannot expand (radius of gyration 32 against 80), Fmax 5 stays in flux (11.6 patches a tick), k 0.25 is poor (nn 0.46 D), and a 2.5 D range cannot expand (29), all as Figs. 3.8–3.12 say; Kill One and One is Born are absorbed. On `formation_goal_obs.nlogo`'s world, goal force 2 brings the formation in 8× faster than 0.25 and compressed (34 against 48); the twelve discs of Fig. 3.25 are crossed by all 50 (674–787 ticks, 606–637 without them), and the wall of Fig. 3.26 is passed round its north end by every particle (8 runs, at the text's size 50 and the measured 45). *Disable One:* the book says goalF must exceed Fmax (the Abandon Goal Force monitor, 1). For a lone neighbour that is exact (it stays at 0.99, leaves at 1.01); in a formation 0–2 neighbours stay at goalF 0.2–0.5 and none at 0.7 or more (8 seeds): the rest of the lattice, leaving, pulls them off, so the monitor's value is a bound that always suffices, not the threshold.

F77. Ch. 3.2.4: the honeycomb traps particles at cell centres, and `honeycomb.nlogo`'s switching frees them by mixing the two rules — 2026-10-25, Confirmed and New. `ch03/README.md`, `checks/ch03.lua`. On a line-for-line copy of the model (201 particles, D 20, G = G_t): with the honeycomb rule alone 29–39% of the particles have two

neighbours under 90° apart at 500 ticks, and 21–38% still at 3000 (particles held at cell centres, as the book says); with the model's switching 6% at 500 and 4–5% at 3000. The book says the model switches periodically between the square and honeycomb lattices. It does not, over time: each particle counts the neighbours it looks at (in a random order every tick) and uses the square rule for 100 and the honeycomb rule for the next 100, so every tick a random half of its pairs feel each rule. The lattice is a hybrid, like neighbours at 1.55 D between √2 and √3 D. A true periodic switch (ours, 100 ticks each) also works, more slowly: 42–53% at 500 ticks, 0–10% from 1800.

F78. Ch. 4.3: uniform coverage reproduces Table 4.2, Fig. 4.19 and William's NetLogo runs; below a mean free path of 17 the runs are noise — 2026-10-25, Confirmed. `ch04/README.md`, `checks/ch04.lua`. On a line-for-line copy of `uniform.nlogo` and `uniform_mfpl.nlogo` (`ch04/coverage.lua`), every run stopped by one rule (doubling blocks of agent steps from 2×10^7, stop within 1%, at most 2×10^8, the book's budget): Table 4.2's 0.027 / 0.026 / 0.016 / 0.004 for 1 / 10 / 100 / 1000 particles come out 0.0272 / 0.0265 / 0.0168 / 0.0045; Fig. 4.17's corners > edges > centre; Fig. 4.19's curves, including one particle's dip (0.0231 at MFP 50, 0.0037 at 65, 0.0184 at 80) where the book says 53–65; MFP 120 equals no mean free path to the last digit (the longest path is 113). William's NetLogo deviations (`ch04/uniform_data.txt`) are reproduced to within a few percent at MFP 17–100 for 1–100 agents and ~10% for 1000 (e.g. one agent 0.0039 / 0.0073 / 0.0231 / 0.0267 against 0.0038 / 0.0072 / 0.023 / 0.026 at MFP 17 / 30 / 50 / 100), and the BPP scene `Uniform_Coverage.lua` reproduces his Android column the same way (one agent 0.0018 / 0.0157 / 0.0367 against 0.0019 / 0.016 / 0.036 at 17 / 50 / 100). Below MFP 17 nothing settles in 2×10^8 agent steps: an agent turning every few steps random-walks between cells, and the deviation is sampling noise falling as ~1/√(steps) (one agent at MFP 4: 0.0009–0.0020 over four seeds, against William's 0.0004, which needs ~10^9–10^{10} ticks).

F79. Ch. 11.8: Table 11.1 reproduced from Maxim's files and simulator; the large environments have not settled — 2026-10-26, Confirmed. `ch11/README.md`, `checks/ch11.lua`. Maxim's simulator and environments (`ch11/maxim/`) and a Lua copy of his C that matches it visit for visit (`ch11/maxim.lua`). The KL of the counts in his own result files gives every entry of Table 11.1 (F80 has the exceptions). Every run of the C draws the same random numbers (the seed time(NULL) is above the generator's limit, so it falls back to (1802, 9373)), and his C built today reproduces his 2009 runs (6×6, 82cell, 84cell, 40 million steps) exactly; his 2008 runs (the other 9 environments, 10 million steps) used a stream we could not recover. One run per entry, as Sec. 11.2.6's ergodicity allows, from 40 million steps, doubling to 160 million until the KL moved less than 5%: 31 of the 60 entries settle, most in the small environments, whose bias is real (7cell 0.0006–0.0010, 4cellRoll 0.0007–0.0009). The rest do not: the large environments' KL is mostly sampling noise and keeps falling with the run length, 2–6× from 10 to 160 million steps (35cell size 30: 0.00067 → 0.00026; 10×10 size 25: 0.00048 → 0.00011), with swings on the way (82cell size 10: 0.00223, 0.00108, 0.00181 at 40, 80, 160 million). For those, the table's values measure the run length as much as the environment; the big, irregular environments (82cell, 84cell, 22cell, 35cell) stay the least uniform.

F82. Ch. 4.1: perfect square and triangular lattices reproduced, with the book's momentum signature — 2026-10-26, Confirmed. `ch04/README.md`, `checks/ch04_perfect.lua`. `formation_perfect.nlogo` as plain Lua (`ch04/perfect.lua`) at its defaults (100 particles, G 1000, p 2.5, D 25). "Perfect" measured by fitting the attributes' ideal lattice (moved, turned, scaled) and asking every particle to be within 0.25 D of its own slot: from the model's start the square forms in 120–200 ticks (9 of 10 seeds) and the triangle in 170–200 (10 of 10), spacing within 0.4% of D, RMS 0.02–0.03 D. Linear momentum is conserved by both (< 10^{-12}); angular momentum by the square only (< 10^{-11}), while the triangle's special cases inject up to 300–850 as the particles sort themselves (PDF 130). One is Born: 57 of 60 clones reached their slots in 10–210 ticks (the other 3 when the next clone arrived). D changed live from 25 to 20 or 30 is followed within 100–500 ticks; shrunk slowly, the square breaks into clusters at D 16.5 and the triangle at 14.5–15, where G 1000 is 4–4.4 × G_t (Eqs. 3.4, 3.5) but just reaches Fmax D^p: a formed lattice holds above G_t (Ch. 3's hysteresis) until the force at D reaches the cap (an earlier note here, "4 G_t", read it wrongly as G_t). `init-m-n` gives distinct slots for 100 square and 127 triangular positions; the 128th particle repeats a triangle slot (the code says "good up to 126"). The Android port matches the model except for details that do not change these results.

F84. Ch. 4.3 and Ch. 11: the mean free path's 0.65 × cell size holds as an upper bound, on the Android rule and on Maxim's simulator — 2026-10-26, Confirmed. `ch04/README.md`, `ch11/README.md`. The book's question (PDF 144–146): the largest mean free path at which one robot, given time, covers as evenly as the 10,000-particle swarm (deviation 0.003). One agent of the Android rule (`ch04/coverage1.c`), MFP 1–100, 5.12×10^9 steps (William's run,

`ch04/coverage1_william.txt`): every MFP 1–28 is below 0.003 (0.0002–0.0008 for 1–14, at most 0.0017 at 15–21), above about 30 worse and no better with time (0.037 at 100). Below about 15 it still improves 2–4 times from 2×10^8 to 5.12×10^9 steps; from 17 up the value is set by 2×10^8. Layout dips at MFP 22–27 and 53–65. Maxim's simulator (F81's fix, seed 1, cell 20, 1.6×10^8 steps, `ch11/ mfp_sweep.lua`): at f = 0.6 every one of his 12 environments is at or below the book's KL target 0.001 (0.00004–0.00086; 84cell and 82cell 0.00107, 0.00124); the largest f that stays below it is 0.6–0.65 in the hardest (4cellRoll, 22cell), larger in the rectangles; above, KL rises to the walls-only values (0.0035–0.014) whatever the run length. On the 3×3 the two simulators agree in f = MFP / cell width (good to 1.0, worse from 1.05, dips just under 1 and at 2.3–2.4, walls only 0.035). Maxim's simulator has a floor the Android rule does not (deviation 0.0033 on the 3×3, KL 0.00005; centre most, corners least): his robot stops a response range short of a wall.

F86. Ch. 4.4: Couette flow's linear profile and conserved momentum reproduced; V/6 holds to a few percent — 2026-10-27, Confirmed. `ch04/README.md`, `checks/ch04_couette.lua`. `kt.nlogo` as plain Lua (`ch04/couette.lua`) and, bit for bit, in C (`ch04/couette1.c`), at its defaults (1000 particles, T 0.010). With the walls moving at ±V the left half of the corridor sweeps down and the right half up, the 13 bars rise linearly (correlation 0.998–0.9995 at the book's 5,000 samples, 0.9994–0.9995 at Fig. 4.22's 15,032 ticks against its 0.99954), and the momenta stay within 3 standard errors of 0. The slope is close to the book's 2V/12 = V/6 but not exactly it: it moves with density and temperature (V = 1: N 500, 1000, 2000 give 0.960, 1.027, 1.075 × V/6; T 0.001, 0.01, 0.04 give 1.050, 1.027, 0.990), and the profile is a slight S, flatter by the walls (end bars 0.96–0.97 V). Steady by 5,000 ticks (100,000 give the same). F85 has what the `.nlogo`'s walls do to the two figures.

F92. Kerr's own 2004 simulator, run unchanged, reproduces his own theory runs; ported to Lua exactly — 2026-10-29, Confirmed. `ch07/README.md`, `checks/ch07.lua`, `ch07/kerr/`. The Java behind the FAABS paper and Kerr's thesis Ch. 5 (William's archive, `code.7.04`), Garcia's DSMC Couette program (`dsmcne`, *Numerical Methods for Physics*, 2000) made 2D, in `ch07/kerr/` with one line changed to take a seed. Run with his drivers' settings it gives his output files' numbers: mean speed +3.7 to +4.0% of √(πkT/2m) (his file +3.7 to +4.0%), density error 0.184–0.192 of his kind (0.186–0.191), profile error 0.160, 0.110, 0.091 of his kind at Mach 0.65, 1.2, 1.792 (0.150, 0.115, 0.088). `ch07/kerr.lua` repeats it operation for operation: four runs from fixed seeds end with the Java's collision counts and states, bit for bit or within 10^{-12} (Lua 5.1–5.4). Without obstacles, and with the three departures of F93 switched off, the chapter's three theories hold on his gas: the mean speed to 0.3%, the density uniform to 2% (N = 10,000), the two-sided profile flat at V to 0.1–1% and the one-sided one linear with slip at both walls (error 20% at Mach 1 to 4% at Mach 8 by the book's measure, falling with V as Fig. 7.7's does).

F96. Ch. 7's corridor sweep (Figs. 7.11–7.13) is Kerr's thesis logs, and his 2005 code, rerun, gives them — 2026-10-30, Confirmed. `ch07/README.md`, `checks/ch07.lua`, `ch07/kerr2005/`. William's archive has Kerr's 2005 robot simulator (KT, Ant and Default = `Simple2`; sonar ring, obstacle courses) with his thesis logs, 100 runs per setting. The logs give the figures point for point: KT's total coverage 76.9% without obstacles and 88.4–89.8% at 10–40%, its shadow coverage 68.2, 81.4, 75.7 and 77.7%, its sweep time 4,848 steps rising to 52,672; Ant 97.5–98.1% total and 100% shadow in 62,000–71,000 steps (126,174 with 5 robots, 57,353 with 30); Default 40.7–45.1% total in 2,000–99,000 steps. Rerun from his seeds (the robots' clock seeds made settable, `ch07/kerr2005_seed.diff`; 225 runs, 5–10 per setting), every setting agrees with his 100 within 3 standard errors in total and shadow coverage (KT's means within 0.3 points at 0–40% obstacles); Default's total sits 1–2 points above his at 0–20% obstacles.

F101. Figs. 7.11–7.13 reproduced from Kerr's own code, 100 runs per point, and the book's version of KT gives the same figures — 2026-10-31, Confirmed. `ch07/README.md`, `ch07/kerr2005_sweep_runs.txt` (William's run), `checks/ch07.lua`. Kerr's protocol on the Lua port (F98): 25 robots at 0–40% obstacles and 5–30 at 20%, 10 courses × 10 runs, his seeds, 5,000 runs. With his code, KT and Ant match his thesis logs at every setting in sweep time, total and shadow coverage (largest difference 1.4 standard errors of 60); Default matches in time and shadow coverage and sits 0.8 points above his logs in total coverage, as his Java did (F96), so his logs came from a slightly different `Simple2`. The book's claim holds: at 20% obstacles KT covers 89.8% in 21,800 steps, a third of Ant's 65,200 (97.9%). KT as the book describes it (`seeds`, `messages`, `wallfit`) gives the same figures within their noise except without obstacles, where it is 20% slower (5,679 against 4,735 steps, 5.2 SE) and covers 1.5 points more; alone, the

book's wall fit does all of it (26% slower, 3 points more; the others change nothing), judging fewer wall hits parallel to the goal (73% against 78%), so fewer get the wall's push toward it.

2. New results

F9. The mean free path in 3D: 0.7720 × the cube edge — 2026-10-17, New. `ch11/README.md`, `ch11/mfp_exact.py`, `checks/mfp_theory.lua`. For cubic cells with face neighbours, the mean distance from a robot uniform in a cube to entering the next is 0.771991712645..., or 0.713804 with the 5% buffer. An integer-relation search on the 60-digit integral finds 90 f = 4 + 17√2 − 6√3 − 7π + 21 ln(1+√2) + 42 ln(2+√3), agreeing to 50 digits (found by search, not proved). Listing 11.4 extended to 3D agrees within one standard error. In BPP's coverage box (edge 26⅔) that is a mean free path of 20.6, or 19.0 with the buffer, against 17⅓ in 2D.

F10. The 2D mean free path in one line — 2026-10-17, New. 12 f = 2 + √2 + 5 ln(1+√2), equal to the book's Eq. 11.3–11.4.

F11. Uniform Coverage: deviation limits by agent count — 2026-10-17, New. At 5×10^7 visits (ticks × agents), mean of two seeds:

MFP	1 agent	10 agents	100 agents
1	0.0046	0.0036	0.0031
4	0.0028	0.0020	0.0024
10	0.0019	0.0023	0.0014
17	0.0021	0.0020	0.0014
30	0.0035	0.0036	0.00087
50	0.0158	0.0131	0.0035
100	0.0368	0.0347	—

MFP 30–100 have converged (seeds agree); MFP ≤ 17 are still falling as 1/√visits and are upper bounds. At equal visits one agent covers as evenly as ten, so what agents buy is time (F8 is the exception, when they crowd).

F12. The book's integrator: energy swings but does not drift — 2026-10-17, New. `checks/ch02.lua`, `checks/monitors.lua`. Semi-implicit Euler keeps energy bounded, with a swing inside each cycle of about Δt·ω: 0.45% for spring1D at Δt 0.001, 16% at the closest approach of the AM 15 orbit at Δt 0.1, 8% for AM 5 passing 1.6 from the sun at Δt 0.001. The drift between the start and end of a run is 10^{-4} or less for the springs and 0.2% for the coarsest orbit. The energy error is first order in Δt (10.2× smaller for 10× smaller Δt). "Constant energy" in the book means no drift.

F13. Ch. 3's obstacle field, measured from the figures — 2026-10-17, New. `lib/world.lua` (`World.CH03_DISCS`). Both panels of Fig. 3.25 give the same 12 disc centres within a pixel, shifted as a group (F40). Scale 2.11 px per patch at 300 dpi from the discs (20 across), confirmed by the formation's 63.3 px node spacing (D = 30). In the model's world (701 × 401, goal at (−175.25, 0); F40) the field's centre is at (−8, −11); the closest gap is 5.2 patches ("narrowly spaced"). Fig. 3.26 is a different field (F37).

F42. spring3D stays in one plane — 2026-10-18, New. `ch02/Spring3D.lua`: from random 3D starts, L keeps its direction to 2×10^{-12} degrees.

F45. η shrinks the lattice when the range reaches second neighbours — 2026-10-18, New. `checks/ch13.lua`, `quadcopter/README.md`. In 2D at the default range 1.5 D, η settles the drones at 0.83 D (power 1) and 0.805 D (power 2.2) instead of D, on all 3 seeds; the particle replica gives 0.82–0.85 and 0.79–0.81, so this is the law, not the drones. η damps the nearest neighbours' repulsion near D far more than the attraction of the second neighbours, which come inside the range (at √3 a) once the lattice spacing a falls below 0.866 D. Balancing the two in an infinite triangular lattice gives a / D = (1 + c) / (1 + √3 c), with c = 3^((1 − P) / (2 p_η)) for law power P: 0.800 (p_η = 1) and 0.762 (p_η = 2.2); a finite swarm's edge, with fewer second neighbours, sits a little wider, as measured. Keeping the

second neighbours out restores the spacing: range 1.25 D gives 0.986 D on the drones (1.001 D as particles), with the best lattice of any run (Delaunay CV 0.069 against LJ's 0.092 and the baseline's 0.11). The 3D triangle's default range is already 1.25 D, and it settles at 1.01 D.

F46. The stop rule needs η — 2026-10-18, New. With the sign flip, the summed force is almost never small: a stop threshold of 0.01 or 0.03 (AP units; in 2D the force just either side of D is ±0.27 per neighbour) held 0.04–0.3% of drone-ticks in 2D and none in 3D, and the swarms jittered on as before. With η it holds every drone once settled (residual speed 10^{-4} m/s in 2D, 10^{-9} m/s in 3D), with the same settling time and lattice as η alone, so it does not freeze the swarm before it has formed.

F47. η softens the lattice: smooth flight, but it gives under load — 2026-10-18, New. With η (power 1) the force's slope at D is F(D)/D: 0.0055 per AP unit in 2D and 0.0085 in 3D, against LJ's $c \cdot p/D^2 = 0.024$, so the lattice is a quarter to a half as stiff (the plain law has no slope at D, only a step). Under the same load η therefore needs about four times LJ's strain. Crossing the default obstacle field after the moving goal (640 s, 2 seeds), η's drones fly smoothly (acceleration below 0.1 m/s² throughout, mean speed 0.10 m/s against the jittering baseline's 0.20 / 0.40 m/s), lose none, and travel slightly farther than the baseline but less far than LJ (swarm centre at 632 s: −46 m against −43 and −50 in 2D; −47 against −41 and −51 in 3D). Under the goal's constant pull their bonds shrink to 0.85–0.92 D in 2D and 0.94–0.95 D in 3D, where LJ's stay at 0.97–1.0. η at the law's own power (2.2 in 2D) settles no better than power 1 and leaves the lattice smaller (0.87 D at range 1.25, still creeping outward after 170 s), so power 1 is the better default.

F49. η has two resting states: switched on after forming, the lattice stays at D — 2026-10-18, New; found by William in the GUI. With η on from the launch, the 2D swarm settles at 0.83 D (F45). Switched on once the plain law has spread the swarm (60 s after AP engages, 3 seeds), the same drones settle within about 2 s at 0.96–0.97 D in 2D, and at 1.00–1.01 D in 3D with bond CV 0.035–0.055, the most regular Newtonian lattice measured (η from the start 0.063, LJ 0.044, plain law 0.083). The particle replica does the same (0.988 D; `checks/ch13.lua`). Both states are equilibria: from a compact cluster the expansion passes through spacings below 0.866 D, where second neighbours are in range and η's balance stalls it; a lattice already near D has them out of range, and η only removes the dither. William's reading: it is annealing, with η as the cooling — the plain law's dither (Ch. 13's "jumpiness", PDF 449) is the heat that lets the lattice reach D, η from the start is a quench into the compressed state. Next: a cooling schedule (ramp `etaPower` 0 → 1) and the fastest cooling that still reaches D.

F50. η in a band tied to D: no trap, a stiffer lattice, and a stiffness limit set by the drones — 2026-10-18, New; William's idea. Damping only within $|D - r| < w \cdot D$ (`etaWidth`; w = 1 is Eq. 13.9) leaves the far forces alone, so from the launch the 2D swarm no longer stalls: bonds 0.83 D (w 1), 0.85 (0.5), 0.98 (0.25), 0.999 (0.15) on 50 drones, 3 seeds each; the particle replica reaches D for w ≤ 0.2, as the F45 balance predicts (no trap for w below about 0.26). w 0.15 gives the best 2D lattice measured (Delaunay CV 0.064; LJ 0.092). Too narrow brings the jitter back: 2D at w 0.1 crashed 4 drones, and 3D at w 0.15 kept jittering in 2 of 3 runs, at 0.1 fully (0.32 m/s). The edge falls at the same slope of the damped force at D, F(D)/(w·D), in both dimensions: 0.034–0.037 per AP unit settles, 0.055–0.057 does not (LJ's slope is 0.024). So the band can be set from the law and the separation, w ≈ F(D)/(0.04 D): 0.14 in 2D, 0.21 in 3D. One w = 0.2 works in both: 2D settles (by the acceleration test; formed at 76–96 s, F52) at 0.994 D (Delaunay CV 0.077), 3D at the last launch wave at 1.01 D (bond CV 0.054), no crashes. Crossing the obstacle field after the goal (2 seeds), w 0.2 holds its bonds at 0.93–0.94 D (2D) and 1.00 D (3D), where w 1 gave 0.85–0.86 and 0.94–0.95 and LJ 0.97 and 0.99, but it is slower: swarm centre at 632 s −43 m in 2D and 3D (w 1: −46, −47; plain law −43, −41; LJ −50, −51). The stiffer lattice threads the columns no faster than the jittering one; LJ is both stiff and fluid.

F51. Ch. 13.4.1.1: adaptive friction damps the dither but cannot stop it, and alone it freezes the swarm — 2026-10-18, Confirmed in part (PDF 463–465), New in part. `quadcopter/README.md`. Eq. 13.23 per axis on the drones' AP velocity (`muMore`, `muLess`), plain Newtonian law, from the launch, 3 seeds. The book's ratio condition holds: at kMore/kLess 2.5 and 10 (kLess 0.02) the swarm keeps moving (2D 0.12 and 0.04 m/s, 3D 0.22 and 0.08 m/s); at 25 it falls below 0.1 m/s² in 6 s (2D) and 30–32 s (3D). But the drones still cross D 104 (2D) and 220 (3D) times per drone per minute, at 0.014 and 0.023 m/s: the small-amplitude oscillation that the book says remains after adaptive friction and that its static friction removes (§13.4.1.4, PDF 465). Our acceleration test alone calls these runs settled; the crossing count shows they are not. The cost is mobility: that residual dither keeps flipping the commands' signs, so

mu stays at 0.8–0.9 and damps the goal's pull too. Crossing the obstacle field (2 seeds) the swarm's centre reaches only +4.5 m (2D) and +8.3 m (3D) in 632 s, against −43 and −41 m for the plain law: the book's "too much damping impairs reactivity" (PDF 463). With η in its 0.2 D band (F50) the dither is gone, so mu decays (0.08–0.11 settled, 0.12–0.26 while following the goal): by the acceleration test settling looked a little faster (5 s 2D, 29–30 s 3D), but the lattice forms later in 2D (91–117 s against 76–96 s, F52), the lattice the same (0.993 D, Delaunay CV 0.082; 3D 1.01 D, bond CV 0.054), and the crossing slightly slower (−40, −38 m against −43, −43 m). In 2D alone it compresses the lattice (0.92–0.94 D); in 3D at ratio 10 it gave the most regular lattice measured (bond CV 0.023) while still moving at 0.08 m/s.

F52. Settling times corrected: when is the lattice formed? — 2026-10-18, Our error, corrected (FIXES 41; found by William in the GUI). The settling times in F44, F50 and F51 came from the acceleration test, which a slow outward creep passes: with η in its 0.2 D band the 2D bonds were still growing from 0.89 to 0.99 D over ~165 s while it said 5–10 s. Timed instead as *formed* (bonds ≥ 0.97 of the spacing and mean speed < 0.01 m/s from then to the end; 285 s runs, 3 seeds, seconds after AP engages):

	2D formed	2D bond CV	3D formed	3D bond CV
η (book band) from the launch	never (0.83 D)	—	42–57	0.063
η, band 0.2 D, from the launch	76–96	0.066	66–154	0.053
η 0.2 D + adaptive friction	91–117	0.066	76–85	0.053
η (book) switched on at 60 s	never (0.96 D)	—	68	0.044
η (book) switched on at 150 s	157	0.046	158	0.020
η 0.2 D switched on at 60 s	88–106	0.063	72–79	0.047
Lennard-Jones	30–35	0.023	51–64	0.015

William's way — the plain law first, η once the swarm has spread to D (F49) — gives the most regular Newtonian lattices, formed about 7 s after the switch; but the plain law needs ~150 s of jitter to reach D in 2D (switched at 60 s, η freezes it at 0.96 D), while in 3D it is there by 60 s. The narrow band is the fastest from the launch in 2D but slower than the book's band in 3D; adaptive friction slows 2D and speeds only the narrow band in 3D. LJ forms fastest in both.

F53. Switching η on per drone by a local rule does not work — 2026-10-18, New (negative). To automate F49, each drone turned η on, and kept it on, by a local test of its own neighbours (one checkbox, no new number). Two tests, 285 s, 3 seeds, 2D and 3D, book band and 0.2 D band: *every neighbour within range at least 0.97 D* never forms a lattice in either dimension (a few drones never pass, keep jittering at 0.06–0.14 m/s and shake the rest; in 2D the early ones freeze patches at 0.93 D); *the mean distance to the neighbours within range at least 0.97 D* passes at once in the launch cluster (neighbours out to 1.5 D average near D), so it is η from the launch again (2D 0.83 D with the book band; 0.2 D band formed at 100–106 s, no better than without the switch). Anything between the two is a new number to tune, so the switch was taken out (HANDOFF §2, simplicity). The local rule that works is η in a band of 0.2 D from the launch (F50, F52); switching η on later is a choice made by hand.

F54. Ch. 13.4.1.4: static friction does not settle the drones — 2026-10-18, New. Eq. 13.25 with the corrected condition $\mu \cdot m \cdot g < |F|$, per axis on the adaptive μ, the plain law without η (the book's whole Pioneer recipe). g is set by the book's own scale, not tuned: g = 9.81 against forces of ~100 at D (Fig. 13.3), so g′ = 0.098 F(D). At muMore 0.2 and 0.5 (muLess 0.02), 3 seeds, no run forms: the drones keep crossing D 123–343 times per drone per minute, as with adaptive friction alone (F51), and in the obstacle-field crossing the swarm is as frozen (centre +3 to +8 m in 632 s). At 10× the book's g′ it is no better (128–367 crossings): blocking an axis zeroes its command, μ decays, and the drone moves again. The book's reasons for it — a minimum wheel speed and poor position estimates (PDF 465) — are absent on these drones (no minimum speed, perfect sensing); it belongs with the ground robots of M2 and with sensor noise (G7). With LJ, F(D) ≈ 0, so it would do nothing.

F55. Ch. 13.4.1.3: the adaptive speed cap is not needed on the drones — 2026-10-18, New. Eq. 13.24 with one reading per tick (the nearest column surface or drone, so no beam count) and the Pioneer's values scaled to the

quadcopter (1.5 / 0.3 m/s, 1.5 m, k_less/k_more 12.5). Crossing a field twice as dense (61–65 columns) after a goal twice as fast (0.4 m/s), 2 seeds, LJ and Newtonian with η: no drone hit a column with or without the cap, and progress was the same within noise (e.g. Newtonian 3D centre at 374 s −12.8 / −14.4 m without, −13.9 / −14.5 m with). The cap never binds: AP's commands stay at 0.1–0.2 m/s, because the obstacle repulsion was already made gentle for these airframes (FIXES 17–23). The only losses are drone–drone collisions in the 2D LJ launch, before the cap could matter. It belongs with faster swarms and with ground robots whose ranging sees obstacles late.

F56. Defaults that work out of the box: η on, band 0.2 D in 2D and 0.5 D in 3D — 2026-10-18, New. William's decision: the drones keep only `etaPower` and `etaWidth`, on by default in the Newtonian scene. Measured with no settings changed (285 s, 3 seeds; launch only, no field): with 0.2 in every mode, the 2D triangle forms at 76–96 s (0.993–0.998 D), the 2D square at 53–170 s (0.97–0.98 D), the 3D triangle at 66–154 s, but the 3D cube keeps jittering in 2 of 3 runs (53 crossings per drone per minute): the 3D law's F(D) is 0.42, so the slope at D, F(D)/(w D), is 0.042, at the edge found in F50. A 0.5 band forms the cube at 80–128 s (0.99 D) and the 3D triangle at 45–50 s (1.008 D, faster than LJ's 51–64 s). So the band follows from the two limits — slope at D below about 0.035 per AP unit (w ≥ 0.16 in 2D, ≥ 0.24 in 3D) and, in 2D at range 1.5 D, w < 0.26 against the second-neighbour trap — and is set per mode: 0.2 in 2D, 0.5 in 3D. The default scene (field, moving goal) loses no drones: swarm centre at 630 s −42 / −44 m in 2D and −45 / −46 m in 3D (plain law −43 and −41, LJ −50 and −51), bonds 0.93–0.98 D under the goal's pull.

F57. Three truly 3D models: a flipping tetrahedron, stable planets, a ringing lattice — 2026-10-18, New. `ch02/README.md`, `checks/ch02.lua`. Two bodies always move in one plane (F42); these need three or more. (a) **Tetrahedron**, masses 1–4 on six springs: an asymmetric top (moments 4390 / 5840 / 7270). Spun 2° off the smallest or largest axis it stays within 3–4° of L for 48 turns; off the middle axis it flips end for end 7 times in 48 turns (5–10 for tilts 0.5–5°, spin 0.3–0.5, k 10–40): the tennis-racket (Dzhanibekov) effect from springs alone, with $|L|$ fixed to 10^{-14} and E to 6×10^{-5}. (b) **Planets**, solar_system's law and units without caps, 4 planets at $15 \times 1.8^{i-1}$ tilted up to 10°: at 1/1000 and 3/1000 of the sun each, all radii stay within 1–3% for 150 outer orbits (3 seeds); at 1/100 one seed of three breaks up by 150 orbits; at 3/100, 1/10, or spaced 1.3× apart, a planet is thrown out. The orbits' tilts drift by degrees as the planets tug on each other. Stability needs light, widely spaced planets, a time step of a few hundred per inner orbit, momentum balanced at the start and circular speeds — and no caps. (c) **Spring lattice**, 3 × 3 × 3 masses with springs on edges and face diagonals (126; edges alone shear flat): it rings without end with P = 0 and L fixed, and with friction settles into the perfect cube (strain 6% to 10^{-8}).

F58. Elliptical orbits: two rules keep several planets stable — 2026-10-18, New (William's question: circles worked, ellipses did not). `ch02/Planets.lua`, `checks/ch02.lua`. Measured with the book's integrator (3 planets, 6 seeds, 20 outer orbits unless stated): (a) **Spacing.** An ellipse swings between a(1 − e) and a(1 + e), so the spacing that works for circles lets elliptical orbits approach or cross. Keeping the *gap* — the outer orbit's nearest point over the inner orbit's farthest point — at the circles' 1.8 (ratio of semi-major axes 1.8 (1 + e)/(1 − e)) holds them: at e 0.3 and 0.5 every orbit stays within 1–6% of its shape for light planets (1/1000 of the sun) and within 16% for heavy ones (1/100), 12 of 12 runs; e 0.7 also holds. At gap 1 (orbits touching) light planets are scrambled (up to 42% past their farthest point) and heavy ones break up in 11 of 12 runs; at circular spacing with e 0.3–0.5, 4 of 12 light runs broke. (b) **Time step.** The nearest-point pass lasts about $\sqrt{(r_p^3/GM)}$, $r_p = a(1 - e)$, much shorter than the orbit. With about 50 steps across it ($\Delta t = \sqrt{(r_p^3/GM)}/50$) one planet keeps its energy within 1% (e 0.5) to 7% (e 0.9) and its orbit does not turn; with 4 steps (e 0.9 at Δt 0.01) the energy swings 88% at each pass and the orbit turns about 1° per orbit; with 1 step, 640% and 157° — and with neighbours, the system breaks. The book's integrator keeps a lone orbit's size even then (it is symplectic), so the damage shows in the energy and the turning, not the size. *Our error, corrected:* the first measure of "broke" used each planet's osculating semi-major axis about the sun, which heavy planets distort by moving the sun; it called stable heavy systems broken. Farthest and nearest distances against the orbit's own a(1 ± e) are the right test.

F59. A pile from a truck: η from the launch is still the best way — 2026-10-19, New (William: in wartime the drones are dropped as a pile, not placed). The default launch is that pile: drones 1 m apart on the ground, so in 2D, where all fly at one altitude, neighbours start ~0.86 D apart (D = 3 m) and the swarm must expand; in 3D the thirds climb to their own levels already ~D apart. Without η the plain law's jitter needs ~150 s to shake out the 2D pile's defects (the

10th percentile of each drone's mean first-shell distance stays ≤ 0.90 D for over 100 s). From the pile, 2D, 280 s, 3 seeds (formed, s / bond CV / drones lost): η from the launch (default) 76, 87, 96 / 0.066 / 0; take-off in thirds, η on 106, 171, 138 / 0.057 / 0; one drone at a time, η on 185, 167, 148 / 0.066 / 0; a per-drone switch latched on once its own first shell (within 1.25 D) averages 0.97 D: 123, 172, 275 / 0.061 / 2 every run; the same switch two-way (η on only while the shell is at D) never forms (bonds 0.95–0.98 D, CV 0.13) / 2 every run, and with the take-off in thirds never (CV 0.13). The two-way switch was meant as local annealing, but drones near the threshold keep turning η off, and each time they jitter again: the heat never falls. Later waves must push into a lattice already damped. In 3D the latched switch formed at 42–53 s (CV 0.060) and the two-way one at 63–277 s, no gain over the defaults. With the drones placed 1.8 m apart (neighbours ~0.96 D), η from the launch forms at 27–33 s, but that is not a pile. So nothing was added (HANDOFF §2): the defaults already get a good lattice from the pile, about 1.5 min in 2D and under 1 min in 3D, and the switch code was removed. LJ forms from the same pile in 30–35 s (CV 0.023): the faster way is in the force law's shape, not in a switch.

F60. The world generator's terrain and water: meanders, the slope rule, and two mistakes of ours — 2026-10-20, New and Our error, corrected. `lib/terrain.lua`, `checks/terrain.lua`. (a) **Meanders.** A river drawn as a sine-generated curve (heading 1.1 sin φ to the valley; Langbein and Leopold 1966) has sinuosity 1/J0(1.1) = 1.390: a 41.7 km test river measures 1.391 and a meander wavelength of 10.9 widths (Leopold and Wolman: 10–14; ours 11). Varying each bend's length by ±30%, so meanders are not all alike, keeps the sinuosity, because a bend's sinuosity does not depend on its length. The presets' rivers measure 1.39–1.40, the city's (only 2.5 meanders long) 1.43. (b) **The slope rule** (no dry slope steeper than 15% along the grid, by lowering only) also makes the river valleys' walls, the ponds' banks and the dry part of dredged cuts, so none needs its own setting. Across a triangle it allows up to √2 × 15% = 21.2%, and four of the five presets reach that on valley walls; that is what "drivable" means here. (c) *Our errors, corrected:* dredging with 1:3 sides after the slope rule left 33% dry banks where a channel ran up into land; sloped cuts now come before the rule and only vertical quay walls after it. Hills drawn as noise of either sign dug an inland hollow 15 m below sea level behind the harbour town (245 nodes); hills now stand on the plain (0 to their height). And a Lua trap worth knowing: `f(t, g(), fn)` passes only g's first result when g returns several (Lua cuts a call's results to one unless it is the last argument). (d) Each preset builds in 0.1–0.4 s in Lua 5.1, and a seed gives identical terrain on Lua 5.1–5.4.

F61. Zones and streets: the published LCZ fractions, and a connected network from one rule — 2026-10-20, Our error, corrected, and New. `lib/town.lua`, `checks/town.lua`. (a) *Our error:* `BOOK.md` §6 gave the open low-rise zone 15–25% built and the sparsely built 5–15%. Stewart and Oke's table (2012, Table 3) gives 20–40% and 10–20% (compact high-rise 40–60% and > 25 m, as we had). `Town.LCZ` now holds the published ranges for all ten built zones, and the buildings (next patch) will be calibrated to them. (b) **One rule makes the network whole:** keep only the streets connected to an arterial. Without it, grid streets cut off by the city's river, stubs at the edges of zones and the port's grids (which stopped 24 m short of their arterial) were left as islands: 15 pieces in the city, 11 in the harbour town, 14 in the port, 1 in the suburb. With it every street is reached from an arterial, which `checks/town.lua` confirms independently by flooding a 4 m raster of the streets (39,441 sample points, all reached). The port's districts were then laid to meet their arterial, so it drops none. (c) **Woods' share** has to be calibrated on the land it applies to: a noise cut taken over the whole area gave woods on 25% and 40% of the open land when 30% was asked; taken over the open land alone it gives 30.0%. (d) A BPP trap: a Lua local named `Terrain` (our terrain module) hides BPP's global `Terrain` mesh class in that file, and the viewer showed nothing; the module is now `Land` there. Zones and streets build in 0.02–0.3 s and a seed gives identical streets on Lua 5.1–5.4.

F62. Buildings calibrated to the published LCZ ranges, and what it took — 2026-10-20, New and Our error, corrected. `lib/buildings.lua`, `checks/buildings.lua`. (a) **Result.** Every built zone covering ≥ 3% of a preset now falls inside Stewart and Oke's ranges, on seeds 1–3 (building fraction / mean height): city compact high-rise 41–45% / 56–64 m (40–60%, > 25 m); compact mid-rise 40–46% / 18–19 m (40–70%, 10–25 m); open mid-rise 23% / 18 m (20–40%, 10–25 m); harbour-town compact low-rise 45–48% / 7 m (40–70%, 3–10 m); open low-rise 22–24% / 5 m in the suburb and the harbour town (20–40%, 3–10 m); sparsely built 12.5–13% / 5 m (10–20%); port large low-rise 33–35% / 8 m (30–50%, 3–10 m); heavy industry 22–23% / 11–12 m (20–30%, 5–15 m). The city's skyline has a heavy tail (median 17.5 m, 99th percentile 80.5 m, tallest 171.5 m, 49 floors), and its mean height falls from 65 m within 150 m of the centre to 31 m and then levels at 18 m, as Schläpfer et al. (2015) describe (heights fall near the centre, then

settle). (b) **The suburb's streets were too sparse.** Cul-de-sacs hung off collectors 400 m apart left block interiors empty: open low-rise 10.6% and sparsely built 5.3% built, a third of the published range. Measuring where the land went (28% street, 47% lots, 25% neither) pointed at the network, not the houses. Local streets 80 m apart between the collectors, half of them cut midway into cul-de-sacs, now carry the lots (55 km of street in the suburb, about 2,300 houses). Cutting needed care: a piece was first assumed to meet a collector at its uncut end, but an end at a zone's edge meets nothing, and those pieces were reversed, trimmed and dropped; locals are now split where they actually cross the collectors. (c) **The gap rule and the setbacks interact.** Back-to-back compact buildings with 1–2 m rear setbacks stood 2–4 m apart, inside the forbidden 0–3 m, so the rule narrowed them and left gaps along the streets; compact zones now have a rear setback of half the minimum gap (1.5 m), and a building breaking the rule gives up depth before width. (d) *Our error, corrected:* lots were first tested against the streets at 3 × 3 points, and on 110–150 m industrial lots whole 20 m streets passed between the points: lots straddled streets and the port's heavy industry read 21.5% built where the honest figure was 18%. The test is now exact (distance from each street segment and turning circle to the lot's rectangle), with water sampled every half cell. (e) The city's centre first sat on its river, which took a quarter of the high-rise zone's land and held it at 36–39% built; it now lies 150 m north. (f) A Lua trap: `goto` does not exist in Lua 5.1 (BPP's cloud build). Lots and buildings take 0.8–3.4 s per preset in Lua 5.1 and are identical on 5.1–5.4.

F63. Mission features and trees: what measuring them corrected — 2026-10-20, New and Our error, corrected. `lib/features.lua`, `lib/vegetation.lua`, `checks/features.lua`. (a) **The storm drain must flood at its outfall.** With the culvert's roof kept under 1 m of cover, a 2 m culvert needs 3 m of ground over its floor; the bay's head is a low shore (~1 m), so the drain leaves the land below sea level: its last 35 m of 458 lie under water, a tide-locked outfall, which real low shores have too. It is recorded (`flooded`) rather than hidden, and matters to the capstone: robots in the drain cannot leave by the sea end. (b) *Our errors, corrected:* the port's ship first sat 2 m off a quay wall that the terrain mesh draws as a slope one 8 m cell wide, and its corner read as aground (−7 m clearance); ships now stand off by a cell and 1 m of fenders (under-keel clearance 3–6 m). A warehouse on a slope had doors 5 m over its lowest ground, which left the uphill door less than 2 m high; doors are now 5 m over the ground at each door. (c) **The gap rule is about obstacles, not pairs.** Container stacks two rows apart are 2.44 m apart, under the 3 m rule, but the row between them closes the gap; the check now merges touching buildings into one obstacle before applying the rule (what the Android app's rule means: gaps between obstacles fit an agent or are closed). (d) **Counts** (seed 1): street trees 7,900 in the suburb, 4,300 in the city, 5,200 in the harbour town; the rural preset has 1,640 lengths of hedgerow and woods as a canopy over 25% of its open land; the harbour town's yard holds 130 stacks in 3 blocks with 6 pockets, the port's 345. (e) **Traps** (open ground no street reaches, 2 m raster): 0.54% of the harbour town's old town and 0.04% of the city core; few, because U-shaped courts open to the rear and back-to-back lots leave 3 m passages. A whole preset builds in 0.4–3.5 s in Lua 5.1, identically on 5.1–5.4.

F64. Rivers: the sine-generated curve was far too periodic — 2026-10-20, Our error, corrected (found by William). `lib/terrain.lua`, `checks/terrain.lua`. F60's rivers followed Langbein and Leopold's sine-generated curve, the *most probable* meander shape, with each bend's length varied by ±30%. Every bend still had the same amplitude and shape, and the rivers read as a pattern: the direction's autocorrelation between points three meanders apart was 0.98. Natural meanders are irregular; Ferguson (1975) found their direction autocorrelograms are *damped* oscillations, and modelled the channel's direction as a damped oscillator kicked by random disturbances (the disturbed periodic model, Ferguson 1976; an AR(2) process with one step a channel width, as fitted to rivers worldwide by Limaye et al. 2021, who note disturbances σ ≥ 0.3 a width are rare). Ours: period 11 × 1.4 widths along the channel, memory falling to 1/e over one meander, the direction smoothed over ~2 widths (no bend tighter than about two widths, Leopold and Wolman), and neck cutoffs where the channel meets itself. Over three 30 km rivers: sinuosity 1.39, wavelength 11.0 widths, autocorrelation three meanders apart 0.02, σ 0.21 a width, no channel touching itself. Steering toward the river's end, the cutoffs and the rounding each take some sinuosity away (1.36 → 1.30), so the disturbances' variance carries a measured gain of 1.5 that restores the mean to 1.4. Downstream every layer moved with the rivers and was measured again: every built zone stays inside its LCZ ranges on seeds 1–3. *Two of our checks were too tight for the new layouts:* F62's heavy-tail test (99th percentile ≥ 4 × the median) rests on a few dozen towers and varies 3.4–4.8 × across seeds; it is now skewness > 3 (measured 4.1–5.7) and tallest ≥ 5 × the

median (6.6–10 ×). And the harbour town's quay now holds two large lots, so the yard takes one and one warehouse is made, as many as there are sheds.

F66. Ground robots freeze in the minimum-speed gap, as the book predicts; Ch. 13's frictions do not get the split law to D — 2026-10-21, Confirmed (book's prediction, PDF 452–453) and New (the numbers). Milestone M2, `ground/README.md`. 50 Pioneer 3-AT robots (book: r_diff 0.3 m, wheels 0 or 0.1–0.5 m/s, u_max 0.5 m/s, Ω_max 0.4 rad/s) under the quadcopter scenes' laws and settings, 2D triangle, compact launch, 180 s, seeds 1–3. A robot moves only while the force on it exceeds friction × 0.1 m/s / (metersPerUnit × apHz) = 0.2 AP units; below that every wheel is in the gap and it stands exactly still (10^{-6} m/s), the book's "agents get stuck in low-energy formations because the forces are too small to move them" (Fig. 13.7). Lennard-Jones, stiff at D, freezes within 1–2% of D: formed at 31.7 s, bond 0.98–0.99 D, bond CV 0.068–0.074 (boats and drones 0.022–0.034). The split Newtonian law has only 0.28 at D, and η (band 0.2 D) damps it under 0.2 out to ~0.14 D from D: frozen at 0.89 D, CV 0.19–0.21, never formed. Without η the robots dither across D at the minimum speed (141–151 crossings per robot per minute), the oscillation the book says the Pioneers had before static friction (E11d). Adaptive friction 0.5/0.02 (Eq. 13.23) cuts that to 44–47; static friction (Eq. 13.25) at the book's scale (g′ = 0.098 F(D)) leaves 38–56 and at g′ = F(D) 75–85: the book's threshold is ~10% of the force at D, and this dither is driven by the full step at D, so static friction cannot stop it. η in a band of 0.05 D freezes at 0.93 D; η at range 1.25 D at 0.94 D with CV 0.12–0.14, the best Newtonian lattice on robots (range 1.25 was the best on drones too, F45). None forms, so none is kept (HANDOFF §2). *The guidance found two rules in its own check* (`checks/vehicle_physics.lua`): without a turn-rate limit of $|u_d| / r_{diff}$ a robot asked to creep spun on the spot at 0.4 rad/s, and a pivot on a wheel stopped in the gap reached 0.56 rad/s until the other wheel was held to 2 r_diff Ω_max. *Since F72 a wheel below the minimum turns at 0.1 m/s instead, and the robots reach D.*

F67. Surface boats follow AP as closely as the drones — 2026-10-21, New. Milestone M2, `boat/README.md`. 50 round ASVs (the book's 0.46 m hull, two Voith–Schneider propellers; mass 12 kg, thrust 12 N, top speed 0.79 m/s chosen), same laws, settings and runs as F66: Lennard-Jones forms in 36–42 s at 1.00–1.01 D (bond CV 0.025–0.030), the split Newtonian law with η in 78–98 s at 0.99–1.00 D (CV 0.057–0.065), against the ideal particles' 31–38 s and 72–116 s at the same lattices, and the drones' 30–35 s (6 lost each, F59's 2D launch losses) and 76–96 s. A holonomic vehicle without a minimum speed needs none of Ch. 13's mechanisms beyond what the drones need.

F68. The cross-vehicle benchmark: the method is vehicle-independent except for a minimum speed — 2026-10-21, New. Milestone M2, `BENCHMARK.md` (reproduce: `probes/benchmark.sh`). The same two tasks and law settings on ideal particles, quadcopters, ground robots and boats, 50 agents, 2D triangle, seeds 1–3. Formation (formed, s; bond): LJ 31–38 / 30–35 / 32 / 36–42 s at 1.00–1.01 D except the robots' 0.98–0.99; Newtonian with η 72–116 / 76–96 / never (0.89 D) / 78–98 s. Goal through the obstacle field (90% past its far edge): LJ 591–666 / 601–638 / never (60–70% past at 960 s) / 698–763 s; Newtonian 688–763 / 679–745 / never (68–80%) / 688–744 s. No vehicle touched a column. The holonomic vehicles match the particles. The robots fail only through the minimum wheel speed (F66): with it set to 0 as a control, the same robots, turning and all, form in 38–39 s (LJ) and 96–134 s (Newtonian) at 1.00 D and get through in 616–680 s and 694–778 s. In the field the minimum turns the slow escape of agents caught behind columns, which particles, drones and boats make, into a permanent stall: a robot whose summed force is under 0.2 AP units never moves again. The drones form the 3D lattices in 45–64 s, the ideal particles (launched at once, not in thirds) in 23–46 s. *The robots' rows are superseded by F72: with small moves made at the minimum speed they match the ideal particles in both tasks.*

F69. Where the world generator's time went — 2026-10-22, New (William: "very very slow"; FIXES 42). Measured inside BPP (Lua 5.1), seed 1, generation + drawing: rural 4.2 s, suburban 8.5 s, city centre 5.5 s, industrial port 4.4 s, harbour town 9.0 s. The time was in a few places, not spread over the language: the lots' street clearance (the exact segment-rectangle distance, for every street segment within 20 m of every candidate lot, five times per lot for the depth search) and the water samples under each lot, 40-60% of the buildings layer; the ground's colouring (a zone lookup per terrain triangle, 0.3 s for 150,000) and three new vectors per triangle when drawing; the woods' canopy (a zone lookup per triangle again). Exact shortcuts (bounding boxes first, a count of wet nodes, shared vertices) give rural 2.9 s, suburban 4.6 s, city centre 3.7 s, industrial port 1.7 s, harbour town 5.6 s (1.5-2.6x), with every world the same to the bit on Lua 5.1-5.4 (`tools/worldprint.lua`). What is left is near the floor for this interpreter: BPP's own

cost of making a mesh (~0.45 s per 150,000 triangles, mostly creating vectors), the zone lookups (the woods' noise, ~1.3 µs a call) and the exact distance tests that cannot be skipped. A tenfold gain needs compiled code for those (C, or LuaJIT). The viewer's repeated rebuilds were a larger cost than any of it (FIXES 42).

F70. Woods as trees: a closed canopy that drones can fly under — 2026-10-22, New (William: "the forests are just blobs, not trees, so drones couldn't go through them"). Zone A was a canopy, a solid volume from the ground to ~20 m. It is now trees at 300 trunks a hectare (William's choice; mature temperate woodland has roughly 200-400), on a jittered grid, crowns of radius 0.15 × height from half the height up: 66,159 trees over 221 ha in the rural preset (300 a hectare), about 16,000 in the harbour town's 55 ha. In the rural woods 1.2% of points are blocked 5 m above the ground (trunks only; the lowest crown starts at 6.4 m) and 73% are under a crown at 14 m, so the canopy is closed above and the floor open between trunks ~5 m apart. The woods' trees come from their own random stream after every other tree, so the rest of each world is unchanged (`tools/worldprint.lua`: identical but for trees, pieces and drawing). Cost, generation + drawing (seed 1, FIXES 42's measure): rural 5.0 s (2.9 s with the canopy, 4.2 s before FIXES 42), harbour town 5.2 s; placing 66,000 trees is ~1.5 s of noise and zone lookups at 270,000 grid points, drawing them ~2 s of BPP's mesh making (1.2 million triangles, lighter than the other trees: 6-sided crowns, 3-sided trunks). Removing the canopy also removed a zone lookup per terrain triangle from every preset's drawing, so the suburb and the city centre got faster (3.7 s and 3.1 s).

F71. Compiled helpers: what they bought, and where the rest goes — 2026-10-24, New (FIXES 43). With BPP's `geo` helpers (src/geo.cpp), seed 1, generation + drawing inside BPP (Lua 5.1, release build, best of two; before, F70's measure): rural 2.9 s (5.0), suburban 2.7 s (3.7), city centre 2.4 s (3.1), industrial port 1.1 s (1.6), harbour town 3.6 s (5.2): 1.3-1.8x, every world the same to the bit (`checks/geo.lua`). Making the woods' 1.2 million triangles fell from ~2 s to 0.18 s and the terrain's grid to 0.03 s; a noise sample costs 0.15 µs instead of 0.94 µs and a zone lookup 0.30 µs instead of 2.1 µs (harbour town). Rural's 2.9 s is now: ~1.1 s Bullet building the search trees (BVH) of the woods' and the ground's meshes in `Terrain:build()` (0.9 µs a triangle, measured on its own), 0.9 s placing the woods' trees (Lua: three random draws at each of 66,000 grid points, the clearance tests, the water and street distances), 0.4 s colouring the ground (a zone lookup per triangle, from Lua), and 0.6 s for the rest. The tenfold gain asked for is not there: the biggest single cost is now Bullet's, for meshes that are only drawn (they are made with no contact response; collisions with trees are the World pieces' job, in Lua), and the next is the placement logic itself, which would have to move to C whole to get much faster.

F72. Ground robots: a small move at the minimum wheel speed, not none, finishes the lattice and gets them through the field — 2026-10-25, New (William: rounding up; "if a robot gets stuck, determine whether there is a gap it can't get through"). Supersedes F66's freeze and F68's robot rows. `ground/vehicle.lua` now turns a wheel asked for less than 0.1 m/s (but not 0) at 0.1 m/s instead of stopping it; where that is against the other wheel's direction the faster wheel is eased, since rounding up alone turned the robot at 0.56 rad/s (the check's limit is the book's 0.4). Same runs as F66/F68, seeds 1–3: formation, LJ formed in 106–175 s at 1.00 D (bond CV 0.021–0.023; before, frozen at 0.98–0.99 D), Newtonian with η formed in 113–159 s at 0.99–1.00 D (CV 0.053–0.076; before, never, 0.89 D); the ideal particles' lattices, later counted as formed because the last corrections are made at 0.1 m/s (crossings 0–9 per robot per minute). Obstacle field: 90% through in 590–673 s (LJ) and 632–666 s (Newtonian), as the ideal particles (591–666, 688–763 s); before, never (60–80% past). No robot stays behind a single column: there the goal's pull and the repulsion balance only on a line, an unstable equilibrium a moving robot slides off. One robot in two of the six runs is left in the field, each time between two columns 2.61–2.65 m apart surface to surface: room for the 0.5 m robot, but the columns' 1.6 m repulsion zones overlap by 0.55–0.59 m, and the goal's pull and both repulsions balance there stably; the robot stands still. `probes/stuck_robots.lua` finds them and measures the gaps; no stuck robot was next to a gap narrower than itself. The ideal particles stop at the same two points to the centimetre (LJ seeds 1 and 3, Newtonian seed 3), so this is AP's equilibrium, not the vehicle's. William's per-wheel rounding without the easing gave the same picture (LJ formed 82–158 s, through 575–649 s; Newtonian 127–168 s, 615–656 s, the same two gaps) but broke the turn limit; scaling both wheels up only when both are under 0.1 m/s kept the limit but left the lattice dithering (mean speed 0.02 m/s, never formed).

F76. Fixed-wing UAVs: the torque law keeps them over the targets; the book's lower turning is not reproduced — 2026-10-25, Confirmed (Fig. 13.13, Fig. 13.16's loops) and New. `ch13/README.md`, `checks/fixedwing.lua`. The

book's aircraft as a kinematic model (`fixedwing/model.lua`): 13 m/s, Ω_max = g sin 20° / (2v) = 0.1290 rad/s (Eq. 13.14; radius 100.7 m, the ~100 m loops of Fig. 13.16; a coordinated turn would give 0.275 rad/s, 47 m, and the factor 2 is unexplained), 10 Hz. One UAV, one attractor, gravity law (Fig. 13.13): directly behind it flies away (1760 m after 120 s); outside the unreachable zones it overflies the attractor again and again (12 passes in 600 s, to 0.04 m); inside one it orbits for ever exactly at the closest reachable distance (50.00 m = 101 − 51). The torque law turns round for an attractor behind. Five UAVs over ten targets, seeds 1–20, 120–600 s: time over the targets' rectangle 12.2% (11.0–14.2) with the torque law against 7.8% (4.7–10.0) with the gravity law, spread 137 m against 147 m, in overlapping loops that cross the targets once per orbit, as the book shows. Mean |Ω| is 0.129 against 0.127 rad/s: both laws turn at nearly the limit, since a loop of the smallest radius is what both fly, so the book's "less turning" is not reproduced; a memoryless torque law with κ shared among the targets turns at half the limit but circles them 205 m out and never crosses them (0% over). Two hazards in the printed law: its repulsion "+κθ" is zero for a neighbour dead ahead (we turn away from the neighbour instead, κ wrap(θ + π)); and with any friction on Ω (Eq. 13.22's (1 − μ_Ω), 0.01–0.05) a UAV leaving with the targets straight behind stays on course in 3–4 runs of 20, the springs to targets either side of its tail cancelling in a stable balance, so the torque law is used as printed (Eq. 13.18, no friction). In one plane the UAVs come within a metre of each other in most runs under both laws (the book does not say); real ones would need height separation. Our choices: G 20,000, μ 0.5, repulsion within 50 m, κΔt/I = Ω_max/π per pair (the book's rule).

F83. Ch. 4.1: Toggle Formation rebuilds a perfect lattice turned 11–15° off the compass; keeping the third law makes the triangle 5–12× slower — 2026-10-26, New. `ch04/README.md`, `checks/ch04_perfect.lua`. After Toggle Formation every particle reaches its slot again (25 of 30 toggles to the triangle in 210–390 ticks, 29 of 30 to the square in 150–2330), but the lattice is turned: triangles +11.3° to +13.2°, squares −11.8° to −15.2°, and it stays turned (50,000 ticks). The formation turns as a whole during the rebuild (seed 1: about 40° to 12°) and stops once every pair is in order; the rules test only which side of each other two particles are, so nothing brings a lattice back to the axes. The rules have no handedness (the problem mirrored in x, positions and m, gives exactly the opposite angle); it comes from `init-m-n`'s numbering of the two lattices' rings. Fitted without turning, the rebuilt lattices look 0.7–0.8 D off, which first read as a failure. The author's question (a), perfect triangles without breaking Newton's third law (the two special cases repulse along the line instead): yes, with angular momentum conserved, but 6 of 10 starts formed in 20,000 ticks, in 850–2230 ticks against 170–200, which supports the book's reason for breaking it. Two lasting failures: a particle left beyond sensing range (1.3–1.7 D) never joins (square, seed 7); and in 4 of 29 toggles to the triangle one particle stays 0.72–0.75 D off (seed 1: a short edge row one place along its slots).

F87. A point mass with the drone's three limits stands in for the drones on the benchmark, at a quarter to a half of the cost — 2026-10-27, New. `lib/pointmass.lua`, `BENCHMARK.md`, `lib/README.md`, `probes/step_response.lua`. G5's point-mass vehicle has a top speed, a top acceleration and a first-order lag, and nothing else. Taken from the quadcopter: its controller's 6 m/s, its tilt limit's g tan 35° = 6.87 m/s², and the lag that best fits its velocity response to AP's legs at 1, 2 and 4 m/s, 0.45 s (RMS 6.5% of the commanded speed; the drone's t63 is 0.4–0.6 s after a ~0.15 s dead time a first-order lag cannot have). On both benchmark tasks and both laws (seeds 1–3) it lands inside the ideal particles' spread and next to the drones': LJ formed in 32–39 s (ideal 31–38, drones 30–35), Newtonian 79–129 s (72–116, 76–96); through the obstacle field in 582–662 s and 689–743 s (ideal 591–666 and 688–763, drones 601–638 and 679–745). Wall time 10–20 s against the drones' 42–48 s for 50 agents, 26 s against 93 s for 100, where the harness's all-pairs AP step is already much of the cost. It has no body, so it says nothing about collisions: at 100 agents the drones lost 14 at the crowded launch, the point masses none.

F88. Couette flow survives the vehicles' lag if kinetic theory keeps its own velocities; colliding the sensed ones freezes the gas — 2026-10-27, New. `ch04/README.md`, `ch04/Couette-Vehicles.lua`. The model run on vehicles through velocity commands (G4), KT sensing actual positions each 5 Hz tick. With KT's own (commanded) velocities in the collisions, point masses with lags of 0.1–0.9 s keep the slope at N 1000 (1.02–1.04 × V/6, ideal 1.02), although a vehicle collides about every 0.25 s; at N 100 to 0.9 s (0.91–0.96, ideal 0.91–0.94), weaker at 1.5 s (0.77). With the sensed velocities in the collisions, a lagging vehicle takes a mix of its old and new velocity into the next collision, which loses energy like an inelastic collision: the gas freezes (mean speed 0.04 √T at N 1000) and the profile is gone (0.002). A robot must carry its KT velocity as state, as Kerr's did, not measure it; and lag is not the "extra viscosity" `BOOK.md` §5 guessed.

F89. Kinetic theory on drones needs collision avoidance underneath — 2026-10-27, New. `ch04/README.md`. On 100 quadcopters (velocity mode, t63 0.6–0.7 s) the profile collapses to 0.18–0.21 × V/6, because the drones fly into each other: KT's collisions are virtual and nothing keeps two airframes apart; 40 of 100 were down by 300 s, each with a neighbour 0.16–0.53 m away when it failed. The point masses, which touch nothing, keep the profile with more lag than the drones have, so the loss is the collisions, not the lag. Kerr assumes "the robots have low-level routines that prevent actual collisions" (thesis, printed p. 37) but gives none; the drones show that assumption is needed. Which routine to add is open.

F90. Separating collisions keep kinetic theory pure and save most of the drones — 2026-10-28, New. `ch04/README.md`, `checks/ch04_couette.lua`. William's choice for F89: every collision keeps its random angle but ends with the pair moving apart (the new relative velocity flipped if it would close them), conserving momentum and energy and adding no force; the collision range is set by the drone's lag (closing speed ~2.8 m/s × lag ~0.65 s + airframe ≈ 2.3 m = 5 units, against the model's 1 m). 100 drones, 2,000 s of flight: slope 0.905–0.908 × V/6 (the ideal vehicle at the same settings 0.980–0.986), 8–10 lost; at the model's range 0.77–0.79 and 17–27 lost; with the model's random collisions 0.18–0.21 and 40 lost in the first 300 s. The rest: slow pairs (a separating collision gives back the pair's own relative speed, so a pair drifting together at 0.46 m/s was told to part at 0.09 m/s and touched), and two failures with no neighbour within 1.5 m when they registered (not determined). A short-range bumper would catch the slow pairs; open (F91: it does not; the slow pair was not the pattern).

F91. A bumper does not save the drones; a KT tick longer than their lag does most of the job — 2026-10-28, New (the bumper tried and taken out). `ch04/README.md`. A low-level bumper under KT (pairs within 1 m or 2.5 m and closing told to part) lost as many drones as none (14 and 7, and 15, against 8 and 10 in 2,000 s). Every loss was a contact after a 4–6.5 m/s jump in the command: a drone flies its old velocity for its ~0.65 s lag, ~3 KT ticks at 5 Hz, and closes 2–3 m first. The closing distance in units depends on the KT rate, not the scale, so the rule is that a vehicle can follow kinetic theory only if its lag is shorter than a KT tick: at 1 Hz with the scale raised to 2.5 m/unit (the same speeds in m/s) the drones track their commands (lag 0.02 against 0.31), carry 93–95% of the ideal vehicle's slope (0.888–0.906 against 0.953), and lose 2 and 5 in 4,000 s, about a fifth of the 5 Hz rate per hour. Not zero: collisions are tested once a tick, and a pair closing faster than range per tick can still meet; avoidance inside the flight controller, which Kerr assumes, is the open remedy.

F102. Ch. 7 on BPP's ground robots: robots jam where Kerr's points do not; backing off, with the bump as a KT wall collision, frees them; KT's first runs cover as the port does — 2026-10-31, New. `ch07/README.md`, `ch07/Corridor-Ground.lua`, `ch07/corridor_ground_runs.txt`. Kerr's controllers drive the Pioneer (`ground/`) in his corridor scaled × 3.28 (83 × 417 m, a 5 m sonar, 0.5 m a step), one step a second, the sonar cast through Bullet. His robots are points moving along a sonar ray; a Pioneer is a 0.5 m box that turns as it goes, and a corner catches an obstacle the ray misses (15° off the heading, or the nose while it turns toward a clear heading 30–130° away). A KT robot's velocity changes only in a collision, so it then pushes there for ever. Without obstacles nothing jams: KT covers 76.4% in 4,480 steps (the port's 100 runs: 76.6 ± 3.2%, 4,735 ± 1,100 steps, mean ± sd), the same to the bit with and without the remedy. At 20% obstacles, without a remedy, the same two runs at 25,000 steps had 16 and 19 robots of 25 done, 9 and 6 jammed (none had moved 0.5 m in 200 steps), coverage frozen at 85.9% and 86.9% (shadow 76.2%, 76.5%), the robots driving 42% and 62% of the path they were told. Backing off alone (0.3 m, after 2 s driven but still) did not free a KT robot: it came back to the same corner (13 back-offs in 800 steps). Backing off with the bump taken as a wall collision (his wall rule: a new velocity away from the wall) did (`backOff`, off by default, on in the sweep): course 0, 89.7% total, 84.0% shadow, 17,546 steps (the port's 10 runs on that course: 90.8, 85.1, 21,629); course 1, 91.1%, 79.5%, 17,572 (89.8, 80.6, 31,053); 34–40 back-offs and 240–280 bumps a run (70–85 robot-robot). Each run's coverage is within the port's spread; the full sweep (300 runs, his two experiments, 10 per setting) answers whether the figures survive. Cost: 8.3–8.7 simulated seconds per wall second per core with 25 robots, Bullet and the robots' per-step Lua through luabind (LuaJIT gains 5%), about 290 CPU-hours for the sweep (`ch07/corridor_ground_sweep.sh`).

F105. Figs. 7.11–7.13 on ground robots: KT and Default survive the wheels; Ant covers as well but takes twice as long, so KT's ~90% comes in a sixth of Ant's time, not a third — 2026-11-02, New. `ch07/README.md`, `ch07/corridor_ground_runs.txt` (300 runs, 298 of them William's). The sweep of F102 on the Pioneer, 10 runs per

setting. KT matches the port (F101) at every setting within its noise: 76.9% without obstacles, 89.2–90.5% at 10–40%, 22,500 steps at 20% (the port 21,800); its robots drive 97–99% of the path they are told. Default's time matches; its coverage is 1–4 points lower (shadow 0–8), its robots driving 87–95%. Ant's coverage matches (96.5–98.1%, shadow 100%), but with 25 robots it takes 113,000–142,000 steps against the port's 62,000–72,000, and 40 of its 100 runs reached the 150,000-step limit; more robots help it less than in the port (121,000 steps with 30, 149,000 with 5). Ant turns 90° every third step, 4 s for the Pioneer, and its robots drive 70–78% of the path they are told: without obstacles it still takes 1.7 times the port's steps. Some Ant robots also stay put for 12,000–28,000 steps or more; why is open (taking each back-off as a blocked step, so that his stuck rule turns the robot, did not free one, and was taken out). At 20%, KT's 22,500 steps are a sixth of Ant's 136,900, where the book and the port have a third (21,800 against 65,200); with 5–30 robots a seventh to a sixth, where the port goes from an eighth to a third.

3. Errors and discrepancies in the book

F14. Listing 11.4's printed result — 2026-10-17, Book discrepancy. `ch11/listing_11_4.lua`, `ch11/README.md`. The listing's entry is at $x_2 = 1.001$ (exact mean 0.652501); the text prints 0.6517. With the book's generator (glibc, srandom(1), Lua 5.1) the listing gives 0.652454, 9 standard errors away, but 0.651711 = "0.6517" at $x_2 = 1$, so the text's run most likely used $x_2 = 1$ (inferred).

F15. Ch. 2.3d: the velocity cap's effect on the orbit's shape — 2026-10-17, Book discrepancy. `checks/ch02.lua`. At Vmax 12 the book says the orbit "runs well, with a slight linear-momentum change". It is slight ($|P| \le 3.0$), but the orbit shrinks unmentioned from 15–87 to 15–18: the cap cuts the start speed 15 to 12. solar_system.nlogo caps as we do (scaling the speed, keeping the direction); clipping each component gives 14.9–18.4.

F74. Ch. 3.2.3: Eq. 3.5 does not fit the model's own square lattice — 2026-10-25, Book error. `ch03/README.md`, `checks/ch03.lua`. Eq. 3.5 gives the square lattice's G_t2 = Fmax D^p / (2 + 2√2) = 186.4. In `formation_newton.nlogo` the clusters break up at 1.12–1.52 G_t2 (median 1.18, 6 seeds). The model renormalises a like pair's distance (r / √2) and also divides its direction by that shorter r, so a like pair's force is √2 longer than the law's; the book's derivation treats it as normal. With the √2 the same argument gives S = 4√2 and G_t2 = Fmax D^p / 4 = 225 = 1.21 × Eq. 3.5, which fits. With the direction from the true distance, as `Formations-Newtonian.lua` and the Java app compute it, the break-up is at 1.02–1.12 G_t2 and Eq. 3.5 holds. The model and the text disagree; the triangle (Eq. 3.4) is unaffected.

F80. Table 11.1: one misprint, mixed run lengths — 2026-10-26, Book error. `ch11/README.md`, `checks/ch11.lua`. 10×10 size 20 is printed 0.00009; Maxim's file gives 0.00053. The text says 40 million steps (PDF 383), but only 6×6, 82cell and 84cell come from 40 million; the other 9 environments come from 10 million (22cell size 20 is 0.00188 at 10 million, the printed value, and 0.00050 at 40 million in another of his files). Three values are cut rather than rounded to five places (0.0000953 printed 0.00009, 0.0006773 as 0.00067, 0.0005972 as 0.00059).

F93. Kerr's theory numbers are his own error measures on a gas that misses the theory in three ways his code does not show — 2026-10-29, Book error. `ch07/README.md`, `ch07/kerr.lua` (fixes `iso`, `walls`, `garcia`, `rest`), `ch07/theory_runs.txt`, `checks/ch07.lua`. The paper and the book define the error as |theory − simulation| / theory (FAABS p. 121, PDF 239); his drivers divide the speed error by the fastest single speed met in the runs (about 4 × the mean), the density excess by N − N/C, the profile error per column by the largest error possible there. By the book's measure the same runs are 3.75% (speed), 36% (density) and 34–61% (profile, his window of steps 1–2,000) off, where his measure reads ~0.8%, 0.19 and 0.09–0.16. Read from his Java and measured (3 seeds; F92's settings): (1) his gas runs hot, $\langle v^2 \rangle = 1.084 \times 2kT/m$ with Maxwell's shape kept, from three causes that act together: the collision angle drawn as $\cos\theta = 1 - 2U$ from the x axis (Garcia's 3D draw without its azimuth), the box's walls at y, and n^2 candidates with c_r,max raised inside the loop (Garcia: n(n − 1), after the cell). Each put right alone leaves +2.1%, +2.3% and +3.6%; all three, +0.28% and $\langle v^2 \rangle$ 1.003. In his units every temperature gives the same run, scaled, so his file's spread over T is his clock seeds. (2) The density box's walls at y send every particle that strikes them toward x = L (vx = +Rayleigh) and put any that leave through y = L back at y = 0: the gas piles into one corner (3.4 × N/C there at N = 10,000). Garcia's wall there makes it uniform (1.9%). (3) The profile's channel is open along the flow: its cells cover 10 L, the gas drifts on with the wall (two in five particles past 10 L by step 500, all by 5,000) and the rest

are sorted into the last row; the profile across the channel is still measured, but over steps 1–2,000 from Garcia's start gradient (−V to +V, in the one-sided runs too), before it has settled: settled (steps 5,001–10,000) the book's error is 20, 15, 13, 11% at Mach 1–4 and 8.3–3.7% at 5–8. The FAABS paper's "3,000 particles … 50,000 time steps" (p. 123) and "the original algorithms from Garcia [10] also have error … slightly below 20%" are not what his drivers ran (500 particles, 2,000 steps; Garcia's original not measured there).

F94. Figs. 7.5–7.8 are the journal paper's figures, from a later simulator with other settings than the book gives; its obstacles are not explained — 2026-10-29, Book discrepancy, Open in part. `ch07/README.md` (the figures' points). William's archive has the journal paper's sources (`journal_wes/`) with gnuplot's PostScript of each figure (`ktunir.2`, `ktv.2`, `ktvy.2`, `actual-vd`, `actual-vd-2`), the book's Figs. 7.5–7.8 curve for curve; the points are read from it. They are also Kerr's thesis's Figs. 5.4, 5.6, 5.2 (right). They come from a simulator with obstacles, 300 robots and 8 subcorridors, not in the archive; the thesis's runs behind the same plots used 100 particles for 10,000 steps (Fig. 5.6 right, printed p. 54) and 500 particles for 5,000 steps (Fig. 5.2 right, printed p. 44), where the book says 300 robots and equilibrium after 1,000 steps (PDF 240–242). For the speed plot the thesis says "less than 10%", the book "less than 2%"; its points are 0.64–1.14%, the level of Kerr's measure (F93: his 2004 gas reads 0.8% by it, 3.75% by the book's). Fig. 7.5 at 0% obstacles is 6.1–6.5%, flat in N, unlike his 2004 box (36%) and close to it with the walls put right (5–6% from N = 500, but falling with N). Fig. 7.7's 0% curve has points at V = 1–6 only (35 → 15%; the chapter note's "≈ 7% at V = 8" is not in the figure); it lies between his 2004 code's window and its settled flow. Fig. 7.8 right is flat at 0.90 V (3.58 of 4), where his 2004 code gives 0.98 V after 2,000 steps and 1.00 V settled. Obstacles: 10% of them cost the two-sided flow 6%, 30% 14%, 50% about half. Kerr's gas obstacles in the archive (April 2004, `SquareObstacle`, for the FAABS coverage runs) are squares at rest that thermalise like his box's walls. His 2005 robot code also has circles and triangles, but only for the sonar, and all 50 of its courses are rectangles; the triangle, circle and cup of Fig. 7.2 (August 2004) are from a version not in the archive. In a steady channel the walls must give what the obstacles take, and a particle flying with the flow meets one of side s every s / (−ln(1 − φ)) at coverage φ (280″ at 30% for 100″ squares, less than the channel's width). Measured (`ch07/obstacles.lua`, `theory.lua flow`, added 2026-10-29): Kerr's 2005 generator's layouts in his gas made Garcia's and periodic, V = Mach 4, 8 subcorridors, three rules at the faces (thermal at rest; faces parallel to the flow moving at V, as his robots treat them; a mirror), 3 layouts each. Without obstacles the one-sided profile is Fig. 7.8 left's (slope 0.76 of the theory's against 0.80). With them no rule comes near: at 10% the two-sided flow is 0.14–0.27 V (Fig. 7.8: 0.84 V), the one-sided 0.05–0.17 V (0.35 V), and half the particles move backward; at 50%, 0.02–0.06 V (0.48 V). Kerr's 2005 robot controller (`KTRobot`, run on William's copy, 4 runs) is not the figures' source either: without obstacles its two-sided profile is U-shaped (2.6–3.4 units per step by the walls, 0.9–1.7 between), not flat at 3.6; Kerr wrote as much (email to Diana and William, 19 March 2011): the journal's theory runs were "just in Java", in "toroidal worlds" with "several hundred robots", and "we never reran the experiments on the robot simulator". The two-sided runs were new in August 2006 (his email "Two Sided Couette Results", 22 August 2006, from ISI, with `actual-vd-2.eps`). That simulator is not in the archive. A steady gas cannot pass obstacles as freely as Fig. 7.8's flow does; how those runs measured it is open.

F97. Ch. 7's robot-number sweep ran at 20% obstacles, not 30%, and every point is 10 corridors, not five — 2026-10-30, Book discrepancy. `ch07/README.md`, `checks/ch07.lua`. The book says Exp. 2 (5–30 robots) used 30% obstacles and both experiments five corridors × 10 runs (PDF 246–247). Kerr's driver for Exp. 2 (`Thesis_2.java`) sets 20%, and his logs agree: 25 robots in Exp. 2 give total 89.8%, shadow 81.6% in 20,921 steps, as his 25 robots at 20% in Exp. 1 (89.8, 81.4, 20,635), not at 30% (89.1, 75.7, 24,667). Every setting in his logs is 10 courses × 10 runs. Also not in the book: all KT robots draw from one random sequence (each seeds Java's generator from the clock in milliseconds when created, all in the same millisecond; Ant too), the KT wall speed in these runs is 0.4 (default 2.0), and robots move 6″ a step when the sonar shows the way clear. The chapter note's "KT ≈ 10 000 steps at 0%" (Fig. 7.11) was a misreading of the figure; the log has 4,848.

F98. Kerr's 2005 simulator in Lua, run for run; his KT robot as coded differs from the chapter's text in three ways — 2026-10-30, Book discrepancy. `ch07/README.md`, `ch07/kerr2005.lua`, `checks/ch07.lua`. The port rounds every float operation of his Java to a float and draws Java's own random numbers; five reference runs of his Java (KT, Ant, Default, 0–20% obstacles, to the end or 500–2,000 steps) end in the same state, the same cells covered at the same step: bit for bit, or within ~10^{-12} where Java's and the C library's cos and sin differ in a double's last bit (whole runs

of 22,000–33,000 steps stay within 7×10^{-13}). His Java's reruns of F96 rerun on the Lua (60 runs: KT and Default at every setting, Ant at 0 and 20%; up to 64,552 steps) give his lines exactly: the same sweep times and cells covered. What the book does not say, read from his code (the last three now switches, off = his code): (1) a KT robot always moves the safe distance, 6″ when the sonar shows the way clear; the collisions' velocity sets only its heading, so Listing 7.1's `max(…, 6)` capped at V_max is right (F99); (2) a robot told its new velocity reads the sender's bearing as the velocity's angle and 0 as the bearing (`messages`); (3) the wall's line comes from the first sonars that see anything within 60″ either way round the ring, at the wrong angles, not the leftmost and rightmost within ±45° and 15″ (PDF 232; `wallfit`); and all KT and Ant robots share one random sequence (F97; `seeds`). ε is 45°, the collision cooldown 5 steps. Put right one at a time or together (KT, 25 robots, 20% obstacles, his 10 courses), none moves total coverage (89.5–90.2% against his 89.9 ± 0.4), shadow coverage (81.0–83.0 against 82.2 ± 1.1) or sweep time (19,900–23,900 against 23,200 ± 2,100 steps) beyond its noise: his figures stand for the controller the book describes.

The chapter notes list further suspected errors in the listings (Ch. 3, 6, 7, 9, 13, 18; `BOOK.md` §1 item 11). They are not yet re-checked by measurement and will be entered here as each chapter is ported.

4. Differences between implementations

F16. Uniform Coverage: the wall rule decides the large-MFP results — 2026-10-17, Divergence (kept by decision). `FIXES.md` 36. NetLogo turns when the patch 1 ahead or either patch 2 away at ±45° is wall; the Android app, and BPP from it, turns only on touching the boundary. With NetLogo's rule swapped into BPP (one agent, 10^7 ticks), MFP 30 / 50 / 100 give 0.0074 / 0.0227 / 0.0271, reproducing NetLogo's own 0.0072 / 0.023 / 0.026. The Android rule gives 0.0035 / 0.0158 / 0.0368: better at every useful MFP, worse only at 100. NetLogo is not maintained; the Android/BPP rule stays.

F17. Uniform Coverage: the two known Android bugs barely matter — 2026-10-17, Divergence. `FIXES.md` 4–5. The one-sided neighbour cone (0–30° for ±15°) and the dropped ninth move were real, but move the deviation about 3%, so BPP and Android agree (MFP 30, 100 agents: 0.00087 against 0.00085).

F18. Earlier NetLogo and Android numbers at small MFP came from longer runs — 2026-10-17, Open. William's old values at MFP ≤ 17 (0.00027–0.0016) sit below our 5×10^7-visit upper bounds, so those runs were longer. Their lengths were not recorded.

F19. Zarzhitsky's map format, kept by the Android port — 2026-10-17, Confirmed. `lib/world.lua` (`writeCPT`). A text file of nx lines of ny characters, `' '` open, `'b'` blocked. His PlumeGen, rebuilt from source, loads a map written by our generator and reports the same coverage (3.6%). Its reader as shipped wants a square of at most 1024; ours writes any size.

F48. `formation_newton.nlogo`, the book and the BPP port use different defaults — 2026-10-18, Divergence. The law is the same in all three (`netlogo/README.md`). The model's sliders are G 310, power 2, Force_Maximum 1, Desired_Separation 30, Friction 0.15, Time_Step 1, 50 particles; the book's text gives G 400 for the same model (`book/ch03.md`); both are above the clustering threshold G_t = 259.8, so the model clusters by design. The BPP ports take their 2D defaults from the Android app's `MyPanel.java`: G 1500, power 2.2, forceMax 1.5, D 50, friction 0.6, which is below its G_t (2368). Ch. 13's damping is in none of them.

F81. Maxim's simulator: the robot can get stuck on a wall — 2026-10-26, Divergence (kept: his simulator as it is). `ch11/wedge.lua`, `ch11/README.md`. The C checks the way ahead with the sensor's end point (float)(cos h + x), rounded once, and moves to x + (float)cos h, which rounds cos h first; the two differ in the last bit, so a move can land exactly on a wall the sensor found clear (4cellRoll size 10, step 6,882,789: sensor at x = 119.999992, robot moved to 120.0). On a wall every sensor line touches it except lines along it, so the robot creeps along the wall on headings within ~10^{-5} rad of it, or makes no progress at all. It happened in 6 of our 60 runs (4cellRoll 10, 4cellL 10, 15cell 20, 35cell 10, 22cell 15, 82cell 15, at 6.9 to 141 million steps; about once per 10^9 steps), never in Maxim's files. The stream is fixed, so a run sticks at the same step every time; those entries end at the last sample before it. Moving to the point the sensor tested would fix it but change every run after the first step that differs.

F85. `kt.nlogo`'s walls emit hotter than the Rayleigh they draw; the book's Figs. 4.23 and 4.24 fit walls that thermalise only particles moving into them — 2026-10-27, Divergence (Kerr's rule, `inward`, the default since 2026-10-28, William; the `.nlogo`'s wall a switch). `ch04/README.md`, `checks/ch04_couette.lua`. A particle within 1 of a wall patch is given a new velocity, Rayleigh (s = √T) away from the wall; but the wall zone is about one step deep, so it is often still inside after the move and is drawn again, every tick, until a draw carries it out. The wall therefore emits a size-biased Rayleigh: particles leaving it carry $|v_x|$ = 1.40–1.42 √T and v_x^2 = 2.38–2.44 T, against 1.2533 and 2. The whole gas runs hot (the mean speed away from the walls equals the mean of all; our first guess, that wall particles inflate the average, was wrong). Fig. 4.24 (walls at rest, √T 0.01–0.20, 100,000 ticks, Resample at 50,000): $\bar{v}$ = 1.358–1.363 √T + 0.0002, r = 0.99999 (2 seeds), against the book's 1.2617 √T + 0.0013 and the theory's 1.2533. Fig. 4.23: slopes 1.007, 1.024, 1.016, 1.006 × V/6 at V = 0.5–2.0 (10 seeds, sd ~0.4%), where the book's curve lies on V/6 to a pixel (0.0004) up to V = 1.5 and at ~0.3297 for 2.0; Fig. 4.22's 0.16256 is 0.1646–0.1660 here. Thermalising only particles moving into the wall (`inward`) brings the emission to the Rayleigh ($|v_x|$ 1.27–1.28 √T) and Fig. 4.23 onto the book's: 0.995, 1.005, 0.998, 0.989 × V/6 (0.3295 at V = 2). Its Fig. 4.24 at 1000 particles is $\bar{v}$ = 1.294–1.298 √T + 0.0004–0.0006 (2 seeds); with fewer particles it falls to the book's value: 1.271, 1.256, 1.268 √T at N 30, 100, 300 (√T 0.1, 3 seeds, ±0.01), while the `.nlogo`'s stays at 1.34–1.37 at every N. So the book's runs behave like walls that leave outgoing particles alone, and its Fig. 4.24 like a lower density than 1000 (its scatter, r = 0.9996, also points to fewer particles). `kt.nlogo` is the book's own model: William's NetLogo 6 conversion of the NetLogo 5 file, and the converter rewrote only syntax (`foreach`'s `?`); `go-particles`, with the wall rule, is the NetLogo 5 code unchanged. So the book's figures did not come from this code at 1000 particles; they came from another version of it or other settings, and the book does not say which. Kerr's thesis (2005, checked against the thesis, printed p. 27), the KT simulator behind the chapter, takes the wall from Garcia's DSMC code: "a check is performed to see if the movement would cause an agent-wall collision. If a collision would occur, then the agent's velocity is reset", and the agent is placed where it would have been had it bounced; so it is thermalised once per contact, only while moving into the wall, and never drawn twice, which is `inward`. `kt.nlogo`'s "within 1 of a wall patch" is a different rule. The book says its Fig. 4.24 comes from "the NetLogo experiments" (PDF 154), not from Kerr's simulator, whose own speed results (500 particles × 50,000 steps, 100 × 10,000) report only "less than 10%" error (printed pp. 53–54). So the book's NetLogo runs behaved like Garcia's and Kerr's rule; most likely an earlier `kt.nlogo` had it (our inference). The Android app has the `.nlogo`'s wall and the same slopes (1.012, 1.024, 1.007 × V/6).

5. Our own errors, corrected

F20. "NetLogo's MFP 30 and 50 values come from short runs" — 2026-10-17, Our error. With NetLogo's wall rule BPP reproduces NetLogo's values (F16), so they had converged; a short run can only raise the deviation.

F21. The seed slider would not move — 2026-10-17, Our error. `FIXES.md` 34. Found by William. BPP stores slider ranges as floats and turns them into an integer slider; a maximum of $2^{31} - 1$ overflowed it. Now 0–1000; larger seeds go through `SEED`.

F22. R in Uniform Coverage reset the seed — 2026-10-17, Our error. `FIXES.md` 34. R was BPP's own reload, which resets every slider to its default, so a chosen seed never applied. Uniform Coverage now binds R to a new scatter, and a seed change restarts the scene in every demo.

F23. The world viewer was mirrored top to bottom — 2026-10-17, Our error. Found by comparing with Fig. 3.25. Book y now maps to BPP −z, so a camera looking down −z sees the book's orientation.

F24. Energy "flat" tested as no swing — 2026-10-17, Our error. The first Ch. 2 checks failed seven times on the integrator's normal within-cycle swing (F12); they now test drift and report the swing separately.

F25. The repository ignored `lib/` — 2026-10-17, Our error. The root `.gitignore` rule `lib` (for build output) silently left the new folder out of the first commit; an exception was added.

F26. A test hook replaced the scene's own — 2026-10-17, Our error. BPP keeps one `postSim` callback, so a probe that set its own stopped the simulation. Probes now use `preDraw` and `preStop` (`lib/batch.lua`).

F27. The 3D mean free path estimate — 2026-10-17, refined: an earlier ≈ 0.7717 (4 M samples) is within error of the exact 0.771992 (F9).

F33. The checks assumed `lua5.1` was installed — 2026-10-17, Our error. Found by William (Lua 5.3 only): every check failed for want of the interpreter, and three used `math.atan2`, gone in 5.3. `checks/run_all.sh` now runs them inside BPP, else any Lua; all code runs on 5.1–5.4.

F37. Fig. 3.26 drawn as Fig. 3.25's discs enlarged — Our error, found by William. Obstacles are placed by mouse clicks, so Fig. 3.26 has its own field: six discs fitted to its outline (preset `ch03-wall`); drawn over the figure, they trace it.

F38. Fig. 3.26's discs measure radius 45, the text says 50 — Book discrepancy (small). Radius 95 px at the scale confirmed by the lattice (F13); the field covers 24.8% of the panel, ours 24.6%.

F39. Code against William's instruction went out in a commit of ours — Our error. Another session's `cpt-textron` preset reached master because I staged the folder unread. Removed; diffs are now read first.

F40. Both figures' obstacles shift between panels: cropping — 2026-10-18, Our error (small), settled by the `.nlogo` and Fig. 3.23: each panel crops ~305 patches of a 701 × 401 world, which we took for the world. Pushing moves a heavy (10^5) obstacle ≤ 0.01 patch/tick. Presets use the model's frame.

F41. Ch. 2's starts guessed, then checked against the .nlogo — 2026-10-18, Our error (small). spring2D now starts at random headings and spring1D's light mass on the right, as the models; F3 barely moves.

F95. Listing 7.3's "half-circle" called a bug; in the controller it is made whole by the listing's own assignment — 2026-10-29, Our error (in `book/ch07.md` and `BOOK.md` §1 item 11), corrected. `ch07/README.md`, Kerr's 2005 robot code (`KTRobot.java`, William's archive). The listing draws sin θ ≥ 0 only, but then "choose[s] the assignment of v1 and v2 so that it doesn't result in an actual collision", and Kerr's controller does exactly that (it swaps the two velocities when the new paths cross): the swap turns v_r into −v_r, the other half-plane. What differs from the text's θ ~ U(0, 2π) is cos θ = 1 − 2U, Garcia's 3D formula, which in 2D favours θ near 90° from the robot's heading. In Kerr's 2004 gas the same draw is taken from the channel's x axis (the pair's random order makes the circle whole there); it is one of the three causes of his gas running hot (F93: +2.1% on the mean speed alone, against +0.5% measured earlier for the draw from a particle's own heading).

F99. "Listing 7.1's `max` should be `min`", and "his simulator writes every robot's state to its log" — 2026-10-30, Our error (in `book/ch07.md`, `BOOK.md` §1 item 11 and `ch07/README.md`), corrected by reading and running his code. His KT robot moves the safe distance whatever its velocity's size (F98), and the book says every algorithm's speed is fixed to the same value (PDF 246): `max(…, 6)` with the cap at V_max is that; `min` would slow a robot to its rebound speed (most probable 1.1″ a step) after every wall. And his `Environment.logRobotInformation` builds a line every move but the logger drops it (its level is off): `log/debug.txt` stays empty.

6. How BPP behaves (worth knowing for every port)

F28. Headless results depend on `delayMs` (FIXES 1): its throttle uses wall-clock time; `lib/batch.py` sets it to 0.

F29. R reloads the script and resets every slider unless a scene binds R itself; the toolbar restart restores the sliders after reloading (F22).

F30. One callback of each kind: `v:postSim`, `v:preDraw`, `v:preStop`, `v:onParamChanged`; setting another replaces it (F26).

F31. Slider ranges are floats turned into integers (value / step): keep them well inside $\pm 2^{31}$ steps; values above 2^{24} are not exact (F21).

F32. Scripts run in their own folder in the GUI; scripts in subfolders add `../?.lua` to reach `lib/`.

F34. BPP's Lua depends on the build: 5.1 or 5.2 (`bpp.pro`), not the standalone `lua`: William's is 5.2, the cloud build 5.1 (`bpp -l 'print(_VERSION)' -n 1`). The checks and the generator's stream agree on 5.1–5.4.

F35. BPP runs a scene as soon as it loads; S stops and restarts it. GUI tests that "pressed S to start" were stopping runs (our error).

F36. Huge "off" values make unusable sliders (Force_Maximum 0.1–10^6 could not be set to 1; found by William). Caps now get a tick box and a slider over the useful range. Same root as F31.

F65. A saved Lua path wins over the script's own folder (found by William, 2026-10-20): BPP uses `lua/path` from its settings (`~/.config/bullet-physics-playground.github.io/bpp.conf`, `[lua] path=`) whenever every folder in it exists, and falls back to the default path (the script's folder and its parents, then `demo/`) only when one is missing (`src/viewer.cpp`, `src/prefs.cpp`). Pressing OK in Preferences saves the path shown, which is computed from where BPP was started. With two checkouts, a path saved in one is used by the other, so `require "common"` failed for `batch_repeat` there. Clear the line (or the field in Preferences) to get the default path back.

F100. LuaJIT: 3–16× on Lua-heavy work, 1.2–1.6× on vehicle scenes, the same results — 2026-10-31, New. `probes/luajit_speed.sh`, `probes/worldgen_time.lua`. BPP's own Lua (5.1, Ubuntu 24.04's build) against LuaJIT 2.1 loaded in its place (`LD_PRELOAD`), every ArtificialPhysics scene headless with `SEED=1` and `delayMs` 0, one run each on two cloud cores (wall seconds; vehicle scenes 5,400 frames):

Work	Lua 5.1	LuaJIT	×
Drones, 50 (Newtonian, LJ)	43.0, 44.3	26.9, 28.2	1.6
Ground robots (Newtonian, LJ)	39.2, 33.4	32.2, 28.1	1.2
Boats (Newtonian, LJ)	51.5, 43.6	35.8, 36.0	1.2–1.4
Particle formations (Newtonian, LJ)	15.3, 18.5	9.6, 13.2	1.4–1.6
Uniform Coverage	5.9	1.1	5.3
Ch. 2: Tetrahedron, SpringLattice, Planets	6.8, 16.1, 1.1	1.5, 5.9, 0.5	2–4.5
Couette on vehicles (3,000 frames); UAV targets	3.1; 2.5	1.0; 1.1	2.3–3.2
World generator, five realistic presets (CPU s)	9.4	2.2	4.3
The 18 standalone checks (outside BPP)	~410	67	6
`checks/ch07.lua` inside BPP	68	4	16

Every scene printed the same output under both, and every check passes on LuaJIT (bit-for-bit ones included). Where Bullet's physics is most of the work (the vehicles) the gain is small; where Lua is (coverage, the generator, the ports) it is large. The world generator takes about the same time with or without BPP's compiled helpers `geo` (9.4 s inside BPP, 9.6 s in plain Lua 5.1, 2.5 s in plain LuaJIT). The preload works only for a BPP linked against Lua 5.1; William's (Ubuntu 22.04) links Lua 5.2, whose luabind is built against 5.2, so LuaJIT there would need luabind and

BPP rebuilt against it, for 1.2–1.6× on the scenes he watches. Not done: LuaJIT is used for long runs outside BPP (the Kerr sweep, `ch07/kerr2005_sweep.sh`).

F103. Bullet's ray test is inexact on boxes, and misses long ones from close by — 2026-10-31, New. `v:rayTest` (`src/viewer.cpp`), found by Ch. 7's sonar (F102). About 230 rays 0.05–3 m from a box's face at 0–60°: Bullet 3.24's convex ray cast (GJK) was off by up to 1.6 cm on a 1 m box, 2.2 cm on 10 m, 8 cm on 33 m, 21 cm on 75 m and 50 cm on 140–418 m, and missed 9 rays on the 33 m box, 36 on 75 m and 90–102 on 140–418 m; from 0.25 m off the corridor's 418 m walls it missed every ray, so the robots' sonar did not see the walls. `v:rayTest` hits boxes by the slab test in the box's frame (exact, to 10^{-4} m in every case) and leaves other shapes to Bullet.

F104. Headless `print` is held until BPP ends — 2026-10-31, New. In `bpp -f`, the script's printed lines go to a buffered stream written when BPP exits, so a long run shows nothing until then, and a killed run loses everything it printed (a 2-hour run did). `io.stdout:flush()` does not help (it is another stream). Progress goes to `io.stderr` (`ch07/Corridor-Ground.lua`'s STEP lines); results are printed at the end and the scene exits with `os.exit` (`KERR_EXIT=1`), which writes them.

7. Open questions

- Ch. 7's obstacles and the simulator behind Figs. 7.5–7.8 (F94): what kind of obstacle keeps Fig. 7.8's flow at 77% of V with 30% coverage, when walls at rest would stop it; the 2005 code, if it turns up.
- F18: the limits at MFP ≤ 17, which need about 10^9 visits to resolve.
- The 3D threshold G_t for the cube formation (`BOOK.md` §1 item 9), to measure with a G sweep.
- How to keep η's spacing at D in 2D at range 1.5 D (F45): a shorter range works, as do switching η on after the lattice forms (F49) and a narrow band tied to D (F50). Whether the slope limit (~0.04–0.05 per AP unit) follows from the drones' lag and AP rate, and holds for other apHz. Static friction and the adaptive speed cap with sensor noise (G7) and real ranging; on the drones they are not needed (F54, F55), and on ground robots with perfect sensing static friction does not settle the split law and the cap has no collisions to prevent (F66, F68). And whether the next Ch. 13 mechanisms (adaptive and static friction) settle the drones without η's loss of stiffness (F47).
- Getting a 2D lattice from a pile faster than η's ~1.5 min (F59): LJ does it in 30–35 s, so the lead is the shape of the force law near D.
- Ground robots with a minimum speed (F66, F68): how to make the small moves that finish a lattice and escape a stall behind a column. The threshold is friction × v_min / (metersPerUnit × apHz) in AP force units, so the law's stiffness near D, apHz and friction all move it.
- Ch. 7 on ground robots (F105): why some Ant robots stay put for thousands of steps; the real 25 × 127 m corridor (robots 3× larger relative to the gaps).